STUDENT SUPER-APP · MANIT BHOPAL

Manit Hub

The Complete Developer Study Guide

Every function, every library, every design decision — written the way a top-tier SDE-1 interview would interrogate it.

React 19 + Vite 7 | Express 5 | MongoDB + Mongoose 9 | Socket.IO | JWT + bcrypt | Cloudinary | Firebase FCM | Capacitor 8 (Android) | node-cron

A single codebase shipping a web app **and** a native Android app.
Web → Netlify · API → Render · DB → MongoDB Atlas

What's inside

Read top-to-bottom. Each part builds on the previous one. Interview-style Q&A boxes (blue) and "why not the alternative" boxes (red) are sprinkled throughout — they are the questions that actually get asked.

1 · The product — what Manit Hub is and why it exists

Manit Hub is a **student super-app** for MANIT Bhopal (NIT Bhopal). It folds ~20 campus jobs — buying/selling, notes sharing, CGPA & attendance tracking, timetables, confessions, ride-share, Q&A, events, lost-&-found, real-time chat, campus maps — into one verified, students-only community that runs both on the **web** and as a native **Android** app.

The 30-second pitch (say this in an interview)

"It's a multi-tenant, university-scoped social + utility platform. The frontend is a React 19 SPA built with Vite and wrapped with Capacitor so the exact same build ships to the browser and to the Play Store. The backend is an Express 5 REST API plus a Socket.IO real-time layer on top of MongoDB. Auth is stateless JWT; every account is tied to a verified university email, and a middleware layer guarantees a user can only ever see data from their own campus. Media goes to Cloudinary, push goes through Firebase Cloud Messaging, and time-based reminders run on node-cron."

Why these specific features

Problem on campus	Feature	The interesting engineering bit
"Where do I buy a used cycle / sell my textbooks?"	Marketplace + Offers + UPI checkout	Negotiation state machine; UPI deep-link + QR generated client-side
"Anyone have notes / PYQs for this subject?"	Study Vault	Arbitrary file uploads streamed to Cloudinary; upvotes + comments
"What SGPA do I need next sem?"	CGPA tracker	Pure client-side math, persisted per semester
"Can I skip the next class?"	Attendance tracker	Closed-form "skippable classes" formula
"I want to say something but stay anonymous"	Confessions	Deterministic, non-reversible anonymous handles via SHA-1
"Reach a seller instantly"	Real-time chat	Socket.IO rooms, read receipts, presence

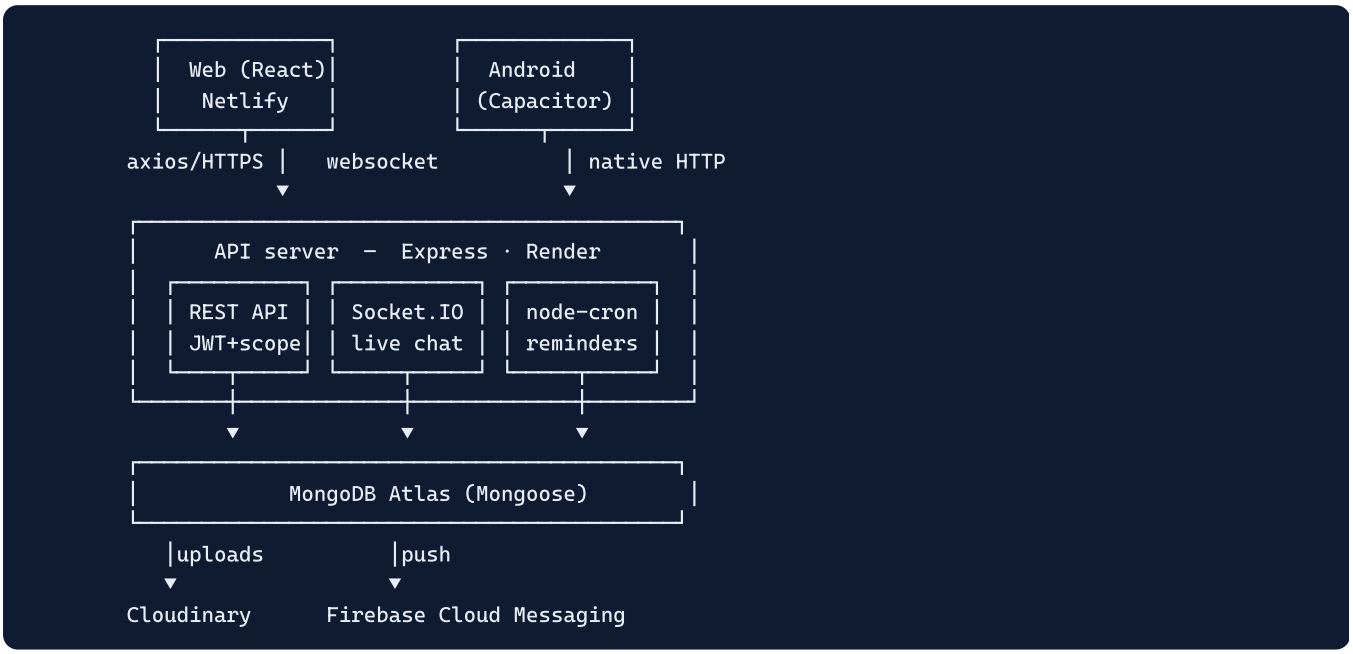
> **Q: Why build a "super-app" instead of separate apps?**
> A: One verified identity, one notification surface, one social graph, and one codebase to maintain. The whole value proposition is *consolidation* — students currently scatter this across WhatsApp groups, Google Forms, and notebooks. Technically it also means shared infra (auth, uploads, chat, push) is written once and reused by every feature.

2 · The 10,000-foot architecture

Manit Hub is a **monorepo** with two top-level apps and a decoupled client/server split:

```
Manit-Hub/
├── frontend/   React 19 + Vite SPA   →  also the Capacitor Android shell
├── backend/    Express 5 REST + Socket.IO   (one .js file per endpoint)
├── docs/        screenshots + (this) developer guide
└── *.apk / *.aab   signed Android builds checked in for distribution
```

The runtime picture



The five non-negotiable invariants

1. **Stateless auth.** The server keeps no session store; a signed JWT carries identity (`id`) and tenant (`university`).
2. **Tenant isolation.** Every query is filtered by `university` . A user physically cannot read another campus's data.
3. **One response envelope.** Almost every endpoint returns `{ success, message, data }` (list endpoints add `meta`).
4. **Graceful degradation.** Push, presence, and image uploads are *progressive enhancements* — if Firebase/Cloudinary creds are missing, the core app still works.
5. **Same code everywhere.** The web bundle and the Android bundle are byte-for-byte the same JS; only the transport (browser fetch vs Capacitor native HTTP) differs.

> **Q: REST and WebSockets in the same server — why both?**
> A: REST is request/response and perfect for CRUD (listings, documents, profiles). Chat needs *server-initiated* pushes — when person A sends a message, person B's screen must update without polling. That's a fundamentally bidirectional, low-latency need, which is exactly what Socket.IO (WebSocket with fallbacks) is for. They share the same HTTP server (`http.createServer(app)`) and the same JWT, so it's one deployment, one auth scheme.

3 · The complete tech stack — every library & why this one

Here is **every** dependency, what it does, and — the part interviewers love — **why it was chosen over the obvious alternative.**

3.1 Backend dependencies

Library	Role in Manit Hub	Why it / why not the alternative
`express` 5	HTTP framework — routing, middleware pipeline, JSON parsing	The de-facto Node web framework. v5 cleans up async error handling. *Alt:* Fastify is faster, NestJS is more structured — but Express's tiny, unopinionated middleware model is exactly what the "one-file-per-endpoint" design needs.
`mongoose` 9	MongoDB ODM — schemas, validation, population, hooks, indexes	Adds a typed schema + lifecycle hooks (e.g. password hashing on `pre('save')`) over the raw driver. *Alt:* the native `mongodb` driver has no schema/validation; Prisma's Mongo support is weaker for nested/embedded docs, which this app uses heavily (comments, reactions).
`jsonwebtoken`	Sign & verify the stateless auth token	JWT lets the server stay stateless — no session table, trivially horizontal-scalable. *Alt:* server-side sessions (express-session + a store) would need sticky sessions or a shared Redis, extra infra this app deliberately avoids.
`bcryptjs`	Hash + compare passwords	Slow, salted hashing designed to resist brute-force. `bcryptjs` is pure-JS so it needs no native build toolchain on the host (Render/serverless friendly). *Alt:* `bcrypt` (native) is faster but needs compilation; `argon2` is stronger but heavier to deploy. **Never** a plain hash like SHA-256 — too fast, no salt.
`socket.io`	Real-time chat + presence over WebSocket	Gives rooms, auto-reconnect, and transport fallback (long-polling) out of the box. *Alt:* raw `ws` is leaner but you'd re-implement rooms, reconnection, and acks yourself.
`multer` 2	Parse `multipart/form-data` file uploads	The standard Express upload middleware. Here it's paired with a **custom streaming storage engine** that pipes straight to Cloudinary (see Part 9). *Alt:* busboy (lower-level — multer is built on it).
`cloudinary` 2	Off-box media storage + CDN + transforms	Offloads binary storage so the API server stays stateless and disk-free. *Alt:* see the dedicated "why not GridFS / S3 / local disk" box in Part 9.
`firebase-admin`	Send FCM push to web + Android	One SDK reaches both browsers and Android devices. *Alt:* raw FCM HTTP v1 (you'd manage OAuth tokens yourself); OneSignal (third-party dependency).
`node-cron`	In-process scheduled jobs (class/event/group reminders)	Zero-infra cron that lives inside the Node process. *Alt:* BullMQ/Agenda need Redis/Mongo queues; OS cron needs a box you control. For a few periodic sweeps, node-cron is the right size.
`helmet`	Sets secure HTTP response headers	One-line hardening (CSP-adjacent headers, no-sniff, etc.). Cheap defense-in-depth.
`cors`	Controls which browser origins may call the API	Configured with a dynamic allow-list (env origins + `*.netlify.app` / `*.vercel.app` + localhost + Capacitor schemes).
`dotenv`	Load `.env` into `process.env`	Keeps secrets (Mongo URI, JWT secret, Cloudinary keys) out of code.

3.2 Frontend dependencies

Library	Role	Why it / why not the alternative

`react` 19 + `react-dom`	UI runtime	Component model + huge ecosystem. v19 ships the modern hooks/Suspense story used here for route-level lazy loading.
`vite` 7	Dev server + build tool	Instant HMR in dev (native ESM) and Rollup-based production builds with manual vendor chunking. *Alt:* CRA is deprecated/slow; Next.js is SSR-first — but this app is a client-rendered SPA that must also become a static bundle for Capacitor, so a pure SPA bundler is the right fit.
`react-router-dom` 7	Client-side routing	SPA navigation with nested layout routes + a protected-route wrapper. *Alt:* TanStack Router — Router 7 is the ecosystem default and integrates with the lazy/Suspense pattern.
`axios`	HTTP client	Interceptors make it trivial to attach the JWT to every request in one place. *Alt:* `fetch` works but you'd hand-roll the auth-header injection and base-URL logic.
`socket.io-client`	Client half of the realtime layer	Must match the server's Socket.IO protocol; gives auto-reconnect that the app leans on (it re-joins rooms on every reconnect).
`framer-motion`	Animations / transitions	Declarative, spring-based motion for the "premium product" feel. *Alt:* CSS transitions (less control over orchestration).
`tailwindcss` 3 + `clsx` + `tailwind-merge`	Utility-first styling + safe class composition	Tailwind tokens encode the MANIT navy/crimson/gold design system + dark mode. `clsx` builds conditional class strings; `tailwind-merge` dedupes conflicting utilities (so `cn()` last-wins correctly).
`lucide-react`	Icon set	Tree-shakeable SVG icons; consistent stroke style. Lazy-chunked separately in the build.
`qrcode`	Generate the UPI payment QR **in the browser**	The QR is built client-side from the seller's UPI ID + price — no server round-trip, no image stored. *Alt:* a server QR endpoint (extra request + storage for an image that's pure function of inputs).
`firebase` (web SDK)	Browser push (FCM) token retrieval	Dynamically imported only when push is configured, so it never bloats the initial bundle.
`@fontsource/*` (Inter, Sora, JetBrains Mono)	Self-hosted web fonts	Bundled into the build so the app renders correctly **offline** inside the Android shell. *Alt:* Google Fonts CDN — would break offline and add a third-party request.
`@capacitor/*` 8 (core, android, app, status-bar, splash-screen, push-notifications)	Wrap the web build as a native Android app	See Part 4 — this is how one codebase becomes two apps.
`leaflet` / `react-leaflet` / `@tmcw/togeojson`	(Map libraries present in deps)	**Honest note:** the shipped `CampusMaps` screen actually renders a Google "My Maps" / Google Maps `output=embed` `<iframe>`, not Leaflet. These libs are carried for an interactive-map iteration. *Why the iframe won:* no API key, no tile hosting/billing, and Google's live POI data for free — see Part 13.

> **Why not TypeScript?** The codebase is plain JS (JSX on the front, CommonJS on the back). Trade-off: faster to write and zero build friction on the backend, at the cost of compile-time type safety. In an interview, the honest answer is: "It's a solo/student-speed project; types would be the first thing I'd add for a team — start with the API response envelope and the Mongoose model shapes."

> **Q: Why `clsx` *and* `tailwind-merge` — isn't that redundant?**
> A: They solve different problems. `clsx` turns conditions into a class string (`clsx('btn', isActive && 'btn-active')`). `tailwind-merge` then resolves *conflicts* within that string (`px-2 px-4` → `px-4`) so the last utility wins — essential when a base component sets `px-2` and a caller overrides with `px-4` . The app wraps both in a single `cn()` helper (`src/lib/cn.js`).

4 · One codebase, two apps — how website & Android stay identical

This is the single most distinctive thing about the project, and a guaranteed interview topic. The website and the Android app run **the same JavaScript bundle**. There is no separate native UI, no React Native, no duplicated screens. The web app *is* the mobile app.

4.1 The mechanism: Capacitor

Capacitor (by the Ionic team) wraps a web build inside a native shell. On Android it creates an Android Studio/Gradle project whose main activity is a full-screen **WebView**. The build pipeline is:

```
npm run android
  = vite build            # 1. compile React → static files in frontend/dist
  + npx cap sync android  # 2. copy dist/ into the Android project's assets
  + npx cap open android  # 3. open Android Studio to build the APK/AAB
```

So the same `dist/` that Netlify serves as a website is the same `dist/` that gets packaged into the `.apk`. **One build, two distribution channels.**

4.2 What actually differs between web and native

The code is identical; only a few *runtime branches* exist, all keyed off one check: `Capacitor.isNativePlatform()`. Here's every place the app cares:

Concern	Web	Android (native)	Where in code
HTTP transport	Browser fetch/XHR via axios → CORS applies	Capacitor native HTTP (`CapacitorHttp.enabled:true`) → bypasses CORS entirely	`capacitor.config.json`
Push notifications	Firebase web SDK + service worker + VAPID key	`@capacitor/push-notifications` → native FCM token	`src/lib/push.js`
Status bar	n/a	Color + light/dark style synced to the theme	`ThemeContext.jsx`
Hardware back button	n/a	Listens for `backButton`: go back in history, or `exitApp()` at root	`App.jsx`
Splash screen	n/a	Native splash (1.2s, MANIT navy)	`capacitor.config.json`
Fonts	Could use a CDN	Must be bundled → `@fontsource` self-hosted	`main.jsx`

The push split is the cleanest example of the pattern:

```
// src/lib/push.js
export async function initPush() {
  if (Capacitor.isNativePlatform()) {
    await initNativePush();   // @capacitor/push-notifications → 'android' token
  } else {
    await initWebPush();      // firebase/messaging + service worker → 'web' token
  }
}
```

Both paths end at the **same backend endpoint** `POST /api/push/register` with a `platform` field, so the server treats every device uniformly (it just stores tokens and multicasts to all of them).

4.3 Why this approach (and why not the alternatives)

> **Why not React Native / Flutter?** Those give truly native widgets but require a **second codebase** (or at least RN-specific components) and can't be served as a website. Manit Hub's whole premise is "ship the web app I already have to the Play Store with near-zero extra code." Capacitor delivers exactly that: the cost is that the UI is a WebView (slightly less "native feel"), which for a content/CRUD app is an acceptable trade.

> **Why not a PWA only?** A PWA can be "installed," but Android Play-Store distribution, reliable native push, splash screens, and hardware-back handling are far smoother through a real packaged app. Capacitor gives the PWA-like single codebase *and* a store-ready binary.

4.4 The CORS-free native trick

On the web, the browser enforces CORS, so the backend keeps an allow-list of origins (including the Capacitor schemes `https://localhost` and `capacitor://localhost` for safety). But with `CapacitorHttp.enabled: true`, the native app's requests go through Android's native HTTP stack, **not** the WebView's fetch — so they aren't subject to browser CORS at all. That's why the README can say "the app talks to the API with no extra CORS configuration."

> **Q: If it's all a WebView, how is it different from just opening the website in Chrome?**
> A: Three things: (1) it's installed/distributed as a Play-Store app with its own icon and offline-bundled assets; (2) it gets native capabilities the browser sandbox won't give a random site as cleanly — native push tokens, status-bar control, splash, hardware-back, app lifecycle; (3) native HTTP sidesteps CORS. The *rendering* is the same web stack, but the *shell and capabilities* are native.

> **Q: How do you keep web and app from drifting apart?**
> A: There's nothing to keep in sync — they're the same artifact. A change to a React screen ships to both the moment you rebuild. The only things that can drift are the *platform branches*, which are deliberately tiny and all funnel through `Capacitor.isNativePlatform()`.

5 · Backend architecture — the one-file-per-endpoint pattern

The backend's defining trait: **every API endpoint lives in its own file**, and each file exports a tiny Express `Router`. There are ~150 such files grouped into 23 feature folders. `app.js` simply `require()`s and mounts each one.

5.1 What a typical endpoint file looks like

```
// backend/documents/createDocument.js  — the WHOLE file
const express = require("express");
const router = express.Router();
const auth = require("../middleware/auth");
const uploadDocument = require("../middleware/uploadDocument");
const Document = require("../models/Document");
const { success, error } = require("../utils/response");
const { awardPoints } = require("../utils/gamification");

router.post("/", auth, uploadDocument.single("file"), async (req, res, next) => {
  // ... validate, create, award points, respond ...
});

module.exports = router;
```

And in `app.js` it's mounted at the feature base path:

```
app.use("/api/documents", require("./documents/createDocument")); // → POST /api/documents
```

5.2 Why split it this way?

Why one file per endpoint instead of a fat controller + routes file?

- **Tiny diffs & no merge conflicts** — each endpoint changes in isolation.
- **Trivial to locate** — the route name *is* the filename (`forum/acceptAnswer.js`).
- **Each file declares exactly its own dependencies** — you can read one endpoint top-to-bottom and understand it fully, which is precisely why this format suits a study guide.
- **Mount order is explicit** — static paths are mounted before `:param` paths so e.g. `/study-groups/upcoming` isn't shadowed by `/study-groups/:id`.

The trade-off is more files and some boilerplate (each repeats the `express.Router()` dance), and shared logic must be factored into `utils/` or a feature-local `helpers.js` / `populate.js` / `serialize*.js`.

5.3 The request lifecycle (every authenticated call)

```
Request
  → helmet            (security headers)
  → cors              (origin allow-list check)
  → express.json()    (parse JSON body)
  → connectDB gate    (await a cached Mongo connection, then next())
  → [route] auth      (verify JWT → load req.user)
  → [route] scope     (universityScope / inline university check)
  → [route] handler   (business logic → success()/error())
  → errorHandler      (last middleware: any thrown/next(err) → JSON)
```

5.4 The DB-connection gate (serverless-safe)

Before any route runs, `app.js` awaits a connection:

```
app.use(async (req, res, next) => {
  try { await connectDB(); next(); }
  catch (error) { next(error); }
});
```

`connectDB` (`config/db.js`) caches the connection **promise**, not just the connection — so concurrent first requests share one in-flight connect instead of opening many. It also refuses a `localhost` Mongo URI in production (a guardrail against misconfigured deploys).

```
let connectPromise = null;
const connectDB = async () => {
  if (mongoose.connection.readyState === 1) return mongoose.connection; // already up
  if (connectPromise) return connectPromise;                            // in-flight: reuse
  connectPromise = mongoose.connect(uri, { serverSelectionTimeoutMS: 30000 })
    .then(c => c).catch(e => { connectPromise = null; throw e; });      // reset on fail
  return connectPromise;
};
```

> **Q: Why cache the *promise* and not just a boolean "connected" flag?**
> A: To avoid a thundering herd. If ten requests arrive while the very first connection is still being established, a boolean would let all ten call `mongoose.connect()`. Caching the promise means the first call starts the connect and the other nine `await` the same promise. On failure the cache is reset to `null` so the next request can retry.

5.5 The response envelope & error handling

Two helpers (`utils/response.js`) standardize every reply:

```
success(res, data, message, status)  → { success:true,  message, data }
error(res, message, status, extra)   → { success:false, message, ...(dev ? extra) }
```

Handlers wrap logic in `try/catch` and forward unexpected errors with `next(err)`. The **last** middleware is the global error handler, which maps `err.statusCode || 500` to a JSON body — so the client always gets JSON, never an HTML stack trace.

> **Q: Why a uniform envelope instead of just returning the data?**
> A: The frontend can write one axios layer that always reads `res.data.data` and surfaces `res.data.message` in a toast on failure — no per-endpoint special-casing. List endpoints extend it with a `meta` object (`page, limit, total, totalPages`) for pagination. Note one small inconsistency worth flagging in interview: a few list endpoints (e.g. `getAllListings`) return `{ success, data, meta }` directly rather than via the `success()` helper — the envelope is a convention, not enforced by a single funnel.

6 · Authentication & authorization, end-to-end

Auth is **stateless JWT**. The token proves *who* you are (`id`) and *which campus* you belong to (`university`). bcrypt protects passwords at rest; a short-lived hashed token drives password reset; the same JWT also authenticates the WebSocket.

6.1 Signup — `POST /api/auth/signup`

Libraries	`jsonwebtoken` (token), `bcryptjs` (hashing, via the User model hook), `mongoose`
Helpers	`resolveUniversityByEmail` , `generateUniqueHandle` , `generateToken` , `success/error`

Workflow:

1. Validate `email, password, displayName` present.
2. Reject if the email already exists (`User.findOne`).
3. **Resolve the university from the email domain.** `resolveUniversityByEmail` looks up a `University` whose `domains` array contains the email's domain; if none exists it *lazily creates* one. This is what makes the platform multi-tenant by simply onboarding a new email domain.
4. **Generate a unique** `@handle` from the email local-part (`generateUniqueHandle` sanitizes to `[a-z0-9_.]` , 3–20 chars, and appends `1, 2, …` until free).
5. `User.create({ … })` — the password is hashed automatically by the model's `pre('save')` hook (it is never stored in plaintext, and the field is `select:false`).
6. Sign a JWT and return `{ user, token }` with status 201.

> **Why derive the university from the email domain instead of a dropdown?** The domain (`@stu.manit.ac.in`) *is* the verification. If you can receive mail there, you're a real MANIT student. It makes signup a single step and guarantees the tenant can't be spoofed by picking the wrong option in a list.

6.2 Login — `POST /api/auth/login`

1. `User.findOne({ email }).select("+password").populate("university", … )` — note `+password` because that field is excluded by default.
2. `user.comparePassword(password)` → `bcrypt.compare` (constant-time-ish, salt embedded in the hash).
3. Return the **same generic error** ("Invalid credentials") whether the email is unknown or the password is wrong — so attackers can't enumerate valid emails.
4. Backfill a handle for legacy accounts created before handles existed.
5. Sign and return a JWT.

6.3 The token — `utils/jwt.js`

```
jwt.sign({ id: userId, university: universityId }, JWT_SECRET,
  { expiresIn: process.env.JWT_EXPIRE || "365d" });
```

Two claims, a **long-lived ~1-year expiry** (configurable via `JWT_EXPIRE`) so users stay signed in until they explicitly log out, HS256 (symmetric — the same secret signs and verifies). Putting `university` in the token lets middleware do a tenant sanity-check without an extra query. Because the auth middleware re-loads the user on every request (see 6.4), a long expiry doesn't weaken revocation — a banned/deleted account stops working on its next call regardless of token lifetime.

6.4 The auth middleware — `middleware/auth.js`

```
const header = req.header("Authorization");           // "Bearer <jwt>"
const decoded = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(decoded.id).select(   // load a lean, fixed projection
  "_id email displayName handle phone bio location avatarUrl university savedListings isAdmin points badge
if (!user) return error(res, "Unauthorized", 401);
// tamper check: token's university must still match the user's
if (decoded.university && user.university.toString() !== decoded.university.toString())
  return error(res, "Unauthorized", 401);
req.user = user;  next();
```

So after `auth` runs, every handler has a trusted `req.user` (including `university`, `isAdmin`, `points`) with no further DB call needed for those fields.

> **Q: You re-fetch the user on every request — isn't a JWT supposed to avoid DB hits?**
> A: A pure-JWT design *can* trust the token alone, but re-loading the user gives you (1) instant revocation/ban (set `isActive=false` and the next request fails), (2) fresh fields like `points` / `isAdmin` without re-issuing tokens, and (3) a guarantee the account still exists. The cost is one indexed `findById` per request — cheap, and a deliberate security-over-microperf trade. If it ever became hot, you'd cache the user in Redis keyed by id.

6.5 Authorization layers

Guard	File	What it enforces
`auth`	middleware/auth.js	You are a logged-in, existing user.
`universityScope(Model)`	middleware/universityScope.js	A specific resource (`:id`) belongs to *your* university. Validates the ObjectId, 404s if missing, 403s on tenant mismatch, and attaches `req.resource` so the handler needn't re-fetch.
Inline owner checks	per endpoint	e.g. "you can't make an offer on your own listing", "only the uploader can delete this document".
`isAdmin`	middleware/isAdmin.js	Gates the moderation queue; must run *after* `auth` (it reads `req.user.isAdmin`).

6.6 Password reset — `forgot-password` & `reset-password`

Uses Node's built-in `crypto` (no extra lib):

1. **forgot:** generate a random 32-byte token (`crypto.randomBytes`). Store only its **SHA-256 hash** + a 30-minute expiry on the user. Always respond with the same "if the email exists…" message (no email enumeration). In non-production the raw token is returned in the body for testing (there is no email-sending integration wired yet).
2. **reset:** hash the incoming token, find a user whose stored hash matches *and* whose expiry is still in the future, set the new password (re-hashed by the model hook), and clear the reset fields.

> **Why store a hash of the reset token, not the token itself?** Same reason you hash passwords: if the database leaks, the stored value is useless — an attacker can't reverse the hash into a working reset link. The raw token only ever exists in the email/response, never at rest.

6.7 Socket authentication

The WebSocket reuses the *same* JWT. On connect, `chat.socket.js` runs an `io.use` handshake middleware that reads `socket.handshake.auth.token`, verifies it, loads the user, and attaches `socket.user`. No valid token → the

connection is rejected. So REST and realtime share one identity scheme. (More in Part 10.)

> **Q: Where is the JWT stored on the client, and what are the implications?**
> A: In **two tiers** (see `src/utils/authStorage.js`): the durable copy lives in **Capacitor Preferences** — native `SharedPreferences` on Android — and a synchronous mirror sits in `localStorage` for the axios interceptor and the socket handshake, which read the token synchronously. **Why two:** Android's WebView flushes `localStorage` to disk lazily and the OS wipes it whenever it kills the app process (swipe-close, low memory), which used to log users out on every restart; Preferences survives that. On the web, Preferences is itself `localStorage`-backed, so the second tier is a harmless no-op there. Trade-off: the token is still JS-readable (XSS exposure), mitigated by React's default output escaping and Helmet headers. The more hardened alternative is an httpOnly cookie (immune to JS access) — but cookies complicate the Capacitor native-HTTP path and CORS credentials, which is why token-in-storage was chosen here.

7 · Multi-tenancy — university scoping

Every piece of user content carries a `university` ObjectId, and every read/write is filtered by it. This is "shared-database, shared-schema, row-level tenancy" — the simplest multi-tenant model, isolated by a discriminator column rather than separate databases.

7.1 How a tenant boundary is enforced (three ways)

Pattern	Example
List queries always include the tenant in the filter	`Listing.find({ university: req.user.university, ... })`
Single-resource access goes through `universityScope(Model)` or an inline compare	`if (listing.university.toString() !== req.user.university.toString()) return error(403)`
Writes stamp the tenant from the trusted `req.user`	`Document.create({ ..., university: req.user.university })`

The tenant value **always** comes from `req.user.university` (server-trusted, set by the auth middleware) — never from the request body. So a client cannot forge which campus it's writing to.

7.2 Even the socket layer is tenant-aware

Before joining a conversation room or sending a message, `chat.socket.js` checks `conversation.university === socket.user.university`. Cross-tenant chat is impossible even over the WebSocket.

> **Why row-level tenancy and not a database-per-university?** One MANIT-only product doesn't need the operational cost of N databases (migrations × N, connection pools × N). A single discriminator column with an index on `university` scales to many campuses on one cluster, and "expand to more NITs" becomes "let users with a new email domain sign up." The risk — forgetting the filter on one query and leaking across tenants — is mitigated by the `universityScope` middleware and the convention that the tenant always comes from `req.user`.

> **Q: What's the failure mode of this design, and how would you defend it?**
> A: The failure mode is a developer writing a query that forgets `university`. Defenses: (1) the shared `universityScope` middleware for `:id` routes; (2) a Mongoose plugin or query-helper that auto-injects the tenant; (3) integration tests that assert cross-tenant reads return empty. In an interview I'd call this out as the main thing I'd harden as the team grows.

8 · The data layer — Mongoose & the models

MongoDB (document store) via Mongoose (ODM). The schema design leans on two MongoDB-idiomatic ideas: **embedding** (sub-documents for things that are always read with their parent) and **denormalization** (storing a computed count next to the data so the common sort/read avoids a join).

8.1 The model catalogue (22 models)

Model	Purpose	Notable fields / design
User	Account + profile + reputation	`password` (select:false, bcrypt), `handle` (unique sparse), `university`, `isAdmin`, `points` / `badges`, `savedListings`, embedded `paymentInfo`, `notificationPreferences`, `privacySettings`
University	Tenant root	`name`, `domains[]` (email-domain → tenant mapping)
Listing	Marketplace item	enum `category` / `condition` / `status`; text index on title+description
Offer	Price negotiation	state enum `pending→countered→accepted/declined/withdrawn`; `seller` denormalized from listing
Document	Study Vault file	`fileUrl/fileName/fileFormat/fileSize`, embedded `comments[]`, `upvotes[]` + denormalized `upvoteCount`
StudyGroup	Branch-wise group	members[], tags, member cap, scheduled sessions, chat links
Confession	Anonymous post	`author` stored but NEVER serialized; embedded `reactions[]` / `comments[]` / `reports[]`; `isHidden` auto-moderation
Conversation / Message	Chat	2-participant invariant; polymorphic `contextType`; `readAt` for receipts
Question / Answer	Q&A forum	upvotes, accepted-answer flag
Event / Ride / LostFoundItem	Campus-life modules	attendees / passengers / status, all `university`-scoped
AcademicRecord	CGPA data	per-semester subjects + grades
AttendanceSubject	Attendance counters	attended/held per subject
TimetableEntry	Weekly class	`dayOfWeek`, `startTime`, `lastReminderDay` (cron idempotency)
Notification	In-app bell	`type`, `read`, polymorphic `relatedId` / `relatedModel`
DeviceToken	FCM registry	one row per device token per user
Friendship	Social graph	`users[2]`, status `pending/accepted`
Report	Moderation	polymorphic target, handled flag

8.2 Three Mongoose techniques worth explaining

(a) Lifecycle hooks — password hashing

```
userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;   // don't re-hash an unchanged pw
```

```
  this.password = await bcrypt.hash(this.password, 10);  // cost factor 10
});
userSchema.methods.comparePassword = function (candidate) {
  return bcrypt.compare(candidate, this.password);
};
```

Hashing lives in the model, so *every* code path that sets a password (signup, reset) is protected automatically. The `isModified` guard prevents double-hashing on unrelated saves.

(b) Denormalized counters

`Document.upvoteCount`, `Confession.reactionsCount` store a number alongside the array. **Why:** the "Top rated" / "Top" sorts run `.sort({ upvoteCount: -1 })` on an indexed scalar instead of computing array lengths across the whole collection on every request. The cost is keeping the count in sync on every up/down action — a classic read-optimization trade.

(c) Indexes

Compound indexes match the real access patterns: `{ university:1, isActive:1 }` on Listing/Document, `{ conversation:1, createdAt:1 }` on Message (chat-history paging), `{ university:1, isActive:1, isHidden:1, createdAt:-1 }` on Confession (the feed query + sort), plus **text indexes** on title/description for search.

> **Q: Why MongoDB and not PostgreSQL?**
> A: The data is document-shaped and read together — a confession with its reactions, comments, and reports is one natural document; a listing with its images array is one document. Embedding avoids joins for the common read. The schema also evolves fast (student project), and Mongo's flexible documents make that cheap. The honest trade-off: cross-entity transactions and relational integrity are weaker, so anything truly relational (offers↔listings↔users) leans on application-level checks and denormalized refs. For a campus social app, document modeling fits; for, say, a banking ledger you'd want Postgres.

> **Q: When do you embed vs reference?**
> A: **Embed** when the child is always read with the parent, is bounded in size, and isn't queried on its own — confession comments, listing images. **Reference** when the child is large, unbounded, or queried independently — Messages reference their Conversation (a chat can have thousands of messages, paged separately), Offers reference Listings.

8 · Data layer (continued) — the complete data dictionary

This is the full schema reference: **every collection, every field, every type, every default, every enum, every index** — enough to rebuild the database from this chapter alone. Collection names are what Mongoose derives from the model name (lowercased + pluralised): `User → users`, `LostFoundItem → lostfounditems`, and so on.

> **Applies to every table below:** all schemas set `timestamps: true`, so every document also carries `createdAt` and `updatedAt` (Date, auto-managed) plus Mongo's automatic `_id: ObjectId`. Those three are omitted from the tables. "→" in the Type column means an ObjectId *reference* to that collection.

How the collections relate

```
universities  (tenant root — every row below carries a `university` stamp)
     |
   users ──────────────────────────────────────────────────────────
     ├─ sells ─────→ listings ←── receives ── offers (buyer + seller → users)
     ├─ uploads ──→ documents            (embedded comments[].author → users)
     ├─ creates/joins → studygroups      (members[] → users)
     ├─ writes ────→ confessions (author hidden) · questions ←── answers
     ├─ chats ─────→ conversations (exactly 2 participants) ←── messages
     ├─ posts ─────→ events (attendees[]) · rides (passengers[]) · lostfounditems
     ├─ owns ──────→ academicrecords · attendancesubjects · timetableentries   (private, per-user)
     ├─ receives ──→ notifications  ·  registers → devicetokens (FCM)
     ├─ befriends → friendships (users[2] + pairKey)
     └─ flags ─────→ reports (targetType + targetId → any content collection)
```

Identity & tenancy

1 · `users` models/User.js

Field	Type	Required / default	Constraints & notes
`email`	String	required	unique · lowercase · must match `^\d{10}@stu\.manit\.ac\.in$` — the scholar-number email is what makes every account a verified student
`password`	String	required	min 8 · `select:false` (never returned by queries) · bcrypt-hashed (cost 10) by a pre-save hook — plaintext never persists
`displayName`	String	required	trimmed
`handle`	String	—	unique + sparse · lowercase · `^[a-z0-9_.]{3,20}$` — Instagram-style @handle; sparse so legacy users without one don't collide on null
`phone` / `bio` / `location`	String	`""`	free text, trimmed
`avatarUrl`	String	`""`	holds the profile photo **as a base64 data URL** — the image bytes live in this field (see §9.5)

`university`	→ universities	required	indexed — the tenant stamp on every account
`isAdmin`	Boolean	`false`	unlocks the moderation queue (`/api/reports`)
`points`	Number	`0`	indexed — the leaderboard sort key
`badges`	[String]	`[]`	earned badge names ("Note Sharer", "Campus Hero"…)
`savedListings`	[→ listings]	`[]`	wishlist / "interested" listings
`paymentInfo`	{ upiId, upiQrUrl }	—	`upiId` matches a UPI regex; `upiQrUrl` is a data-URL image of the seller's QR code
`notificationPreferences`	{ messages, studyGroups, marketplace }	all `true`	per-category Boolean opt-outs
`privacySettings`	{ profileVisibility, showOnlineStatus, allowDirectMessages }	`everyone` / `true` / `true`	`profileVisibility` enum: `everyone · students · private`
`isActive`	Boolean	`true`	soft delete
`isBanned` · `strikes` · `banReason` · `bannedAt`	Boolean · Number · String · Date	`false` · `0` · `""` · `null`	moderation state — strikes accumulate when the user's content is auto-hidden/removed; a banned account is rejected by both the API and the socket
`passwordResetToken` · `passwordResetExpires`	String · Date	`null`	`select:false` — SHA-256 hash of the emailed reset token, never the raw token
`tokenVersion`	Number	`0`	`select:false` — bumped on password reset; JWTs carry it as the `tv` claim, so all older tokens die instantly
`emailVerified`	Boolean	`true`	default **true** on purpose — legacy accounts (pre-verification) keep working; only accounts explicitly created with `false` are gated at login
`emailVerificationToken` · `emailVerificationExpires`	String · Date	`null`	`select:false` — SHA-256 of the 6-digit OTP · 30-minute expiry

Indexes: `email` unique · `handle` unique-sparse · `university` · `points`

2 · `universities` models/University.js

Field	Type	Required / default	Constraints & notes
`name`	String	—	display name ("MANIT Bhopal")
`domains`	[String]	required	unique — email-domain → tenant mapping (`stu.manit.ac.in`); signup resolves the university from the email's domain
`isVerified`	Boolean	`false`	—
`isActive`	Boolean	`true`	—

Marketplace

3 · `listings` models/Listing.js

Field	Type	Required / default	Constraints & notes
`title`	String	required	text-indexed for search
`description`	String	—	text-indexed
`price`	Number	required	min 0 (₹)
`category`	String	required	enum: `Textbooks · Electronics · Furniture · Clothing · Sports · Other`
`images`	[String]	`[]`	base64 **data URLs** — photo bytes live inline in this document
`seller`	→ users	required	—
`university`	→ universities	required	indexed
`condition`	String	`good`	enum: `new · like-new · good · fair`
`status`	String	`available`	enum: `available · sold · reserved` · indexed
`isActive`	Boolean	`true`	soft delete

Indexes: text(`title` , `description`) · `{ university, isActive }`

4 · `offers` models/Offer.js

Field	Type	Required / default	Constraints & notes
`listing`	→ listings	required	indexed
`buyer`	→ users	required	indexed
`seller`	→ users	required	indexed — **denormalized from the listing** so "offers received" is one query, no join
`university`	→ universities	required	indexed
`amount`	Number	required	min 1
`message`	String	—	max 300
`counterAmount`	Number	—	min 1 — the seller's counter (status becomes `countered`)
`status`	String	`pending`	enum (the negotiation state machine): `pending · countered · accepted · declined · withdrawn` · indexed

Indexes: `{ listing, buyer, status }`

Study Vault

5 · `documents` models/Document.js

Field	Type	Required / default	Constraints & notes
`title`	String	required	text-indexed
`description`	String	—	—
`type`	String	required	enum: `Notes` · `PYQ` · `Syllabus` · `Sem Schedule` · `Other`
`branch` / `subject` / `semester`	String	—	`subject` is text-indexed
`year`	Number	—	—
`fileUrl`	String	required	**Cloudinary** `secure_url` — the only pointer to the binary; the file itself is NOT in Mongo
`fileName` / `fileFormat`	String	—	original name · extension
`fileSize`	Number	—	bytes (from Cloudinary)
`uploader`	→ users	required	—
`university`	→ universities	required	indexed
`downloadCount`	Number	`0`	`$inc`'d by `POST /:id/download` for analytics/sorting
`upvotes`	[→ users]	`[]`	who upvoted (prevents double votes)
`upvoteCount`	Number	`0`	indexed — denormalized for the "Top rated" sort
`comments`	[embedded]	`[]`	`{ author → users (required), content ≤500, createdAt }`
`isActive`	Boolean	`true`	soft delete

Indexes: text(`title` , `subject`) · `{ university, isActive }` · `upvoteCount`

Community

6 · **`studygroups`** models/StudyGroup.js

Field	Type	Required / default	Constraints & notes
`name`	String	required	text-indexed
`description`	String	—	text-indexed
`subject`	String	required	text-indexed
`tags`	[String]	`[]`	indexed + text-indexed
`image`	String	`""`	cover image as a base64 data URL
`creator`	→ users	required	a pre-save hook guarantees the creator is always in `members` and dedupes the array
`members`	[→ users]	—	virtual `memberCount` = array length

`maxMembers`	Number	`10`	min 2 — member cap enforced on join
`links`	{ whatsapp, telegram, discord, googleMeet }	—	external collaboration links
`customLinks`	[{ label, url }]	—	anything else
`nextSession`	{ at: Date, mode, location, meetingLink }	—	`mode` enum: `online · offline` — drives the session-reminder cron
`university`	→ universities	required	indexed
`isActive`	Boolean	`true`	indexed
`notifications.sessionReminderSent`	Boolean	`false`	indexed — cron idempotency flag (reset when a new session is scheduled)

Indexes: text(`name` , `description` , `subject` , `tags`) · `{ university, isActive }` · `{ university, isActive, "nextSession.at" }`

7 · `confessions` models/Confession.js

Field	Type	Required / default	Constraints & notes
`author`	→ users	required	indexed — stored **privately for moderation only, never serialized to any client**; anonymity is a serialization rule, not missing data
`university`	→ universities	required	indexed
`content`	String	required	max 1000
`reactions`	[{ user → users, type }]	—	`type` enum: `heart · laugh · wow · sad` — one reaction per user, switchable
`reactionsCount`	Number	`0`	indexed — denormalized for the "Top" sort
`comments`	[embedded]	—	`{ author → users (hidden too), content ≤500, timestamps }`
`reports`	[{ user, reason ≤300, createdAt }]	—	embedded (unlike other content types, which use the `reports` collection)
`isHidden`	Boolean	`false`	flipped automatically when reports cross the threshold
`isActive`	Boolean	`true`	soft delete

Indexes: `{ university, isActive, isHidden, createdAt: -1 }` — exactly the feed query + sort

8 · `questions` models/Question.js

Field	Type	Required / default	Constraints & notes
`author`	→ users	required	indexed
`university`	→ universities	required	indexed
`title`	String	required	max 150 · text-indexed
`body`	String	—	max 3000 · text-indexed

`branch` / `subject` / `semester`	String	—	`subject` text-indexed
`upvotes`	[→ users]	`[]`	—
`upvoteCount`	Number	`0`	indexed (denormalized)
`answersCount`	Number	`0`	denormalized — list view never counts answers
`acceptedAnswer`	→ answers	`null`	set by the question's author; awards +25 to the answerer
`isActive`	Boolean	`true`	soft delete

Indexes: text(`title` , `body` , `subject`) · `{ university, isActive, createdAt: -1 }`

9 · **`answers`** models/Answer.js

Field	Type	Required / default	Constraints & notes
`question`	→ questions	required	indexed — answers are referenced, not embedded (unbounded count, paged separately)
`author`	→ users	required	indexed
`university`	→ universities	required	indexed
`body`	String	required	max 3000
`upvotes` / `upvoteCount`	[→ users] / Number	`[]` / `0`	same pattern as questions
`isActive`	Boolean	`true`	soft delete

Indexes: `{ question, isActive }`

Chat

10 · **`conversations`** models/Conversation.js

Field	Type	Required / default	Constraints & notes
`listingId`	ObjectId	`null`	polymorphic context id — a Listing, LostFoundItem or Ride depending on `contextType` ; `null` for friend DMs
`contextType`	String	`listing`	enum: `listing · lostfound · ride · friend`
`listingTitle`	String	`""`	denormalized display title; empty for friend DMs (UI shows the other participant instead)
`participants`	[→ users]	required	validated to be **exactly 2** · pre-save hook sorts them so [A,B] ≡ [B,A]
`university`	→ universities	required	indexed
`lastMessage` / `lastMessageAt`	String / Date	— / now	denormalized preview + sort key for the inbox list (indexed)

Indexes: `{ participants: 1 }` — deliberately **NOT unique**: a unique index on an array is multikey and would cap each user at one conversation per listingId (the shipped E11000 bug). Duplicate-prevention lives in `createConversation` 's findOne-then-

create.

11 · `messages` models/Message.js

Field	Type	Required / default	Constraints & notes
`conversation`	→ conversations	required	indexed — referenced, not embedded: a chat can have thousands of messages, paged separately
`sender`	→ users	required	indexed
`university`	→ universities	required	indexed
`content`	String	required	the message text
`readAt`	Date	`null`	read receipt — set when the other participant opens the conversation
`type`	String	`text`	enum: `text · system`
`isDeleted`	Boolean	`false`	soft delete

Indexes: `{ conversation, createdAt: 1 }` — chat-history paging

Campus life

12 · `events` models/Event.js

Field	Type	Required / default	Constraints & notes
`organizer`	→ users	required	indexed
`university`	→ universities	required	indexed
`title`	String	required	max 120 · text-indexed
`description`	String	—	max 2000 · text-indexed
`club`	String	—	max 80 · organizing club/society · text-indexed
`category`	String	required	enum: `Cultural · Technical · Sports · Workshop · Talk · Other`
`venue`	String	—	max 120
`startAt` / `endAt`	Date	required / —	`startAt` indexed — drives "upcoming" queries and the reminder cron
`attendees`	[→ users]	`[]`	RSVP list (toggle)
`reminderSent`	Boolean	`false`	cron idempotency — one reminder per event, ~1 h before start
`isActive`	Boolean	`true`	soft delete

Indexes: text(`title`, `description`, `club`) · `{ university, isActive, startAt }`

13 · `rides` models/Ride.js

Field	Type	Required / default	Constraints & notes

`poster`	→ users	required	indexed
`university`	→ universities	required	indexed
`from` / `to`	String	required	max 80 each · virtual `title` = `"from → to"` (used by the shared conversation loader)
`departureAt`	Date	required	indexed
`seatsTotal`	Number	required	1–8 — seats for co-passengers (poster's own seat not counted)
`passengers`	[→ users]	`[]`	join/leave; capacity checked against `seatsTotal`
`note`	String	—	max 300
`isActive`	Boolean	`true`	soft delete

Indexes: `{ university, isActive, departureAt }`

14 · `lostfounditems` models/LostFoundItem.js

Field	Type	Required / default	Constraints & notes
`title`	String	required	text-indexed
`description`	String	—	text-indexed
`kind`	String	required	enum: `lost · found` — did the poster lose it or find someone else's
`category`	String	required	enum: `Electronics · ID & Cards · Books & Notes · Clothing · Keys · Accessories · Other`
`location`	String	—	where lost/found
`images`	[String]	`[]`	base64 data URLs (inline in Mongo)
`reporter`	→ users	required	—
`university`	→ universities	required	indexed
`status`	String	`open`	enum: `open · returned` · indexed
`isActive`	Boolean	`true`	soft delete

Indexes: text(`title` , `description`) · `{ university, isActive }`

Personal academics (private, per-user)

15 · `academicrecords` models/AcademicRecord.js

Field	Type	Required / default	Constraints & notes
`user`	→ users	required	indexed
`university`	→ universities	required	indexed
`semester`	Number	required	1–12

`subjects`	[embedded, `_id:false`]	`[]`	`{ name (required), credits 0.5–30, grade enum A+ · A · B+ · B · C+ · C · D · F }` — SGPA/CGPA math runs client-side on this data
Indexes: `{ user, semester }` **unique** — exactly one document per user per semester (upsert semantics)			

16 · `attendancesubjects` models/AttendanceSubject.js

Field	Type	Required / default	Constraints & notes
`user`	→ users	required	indexed
`university`	→ universities	required	indexed
`name`	String	required	subject name
`attended` / `held`	Number	`0`	min 0 — classes attended out of classes held
`target`	Number	`75`	1–100 — the % the student wants to stay above; the "skippable classes" formula runs on these three numbers
Indexes: `{ user, createdAt }`			

17 · `timetableentries` models/TimetableEntry.js

Field	Type	Required / default	Constraints & notes
`user`	→ users	required	indexed
`university`	→ universities	required	indexed
`subject`	String	required	max 80
`dayOfWeek`	Number	required	0–6, JS convention (0 = Sunday)
`startTime` / `endTime`	String	required	24-h `"HH:mm"`, regex-validated, campus-local (IST)
`room` / `professor`	String	—	max 60 each
`lastReminderDay`	String	`null`	`"YYYY-MM-DD"` of the last reminder sent — the cron's duplicate-stopper
Indexes: `{ user, dayOfWeek, startTime }`			

Infrastructure collections

18 · `notifications` models/Notification.js

Field	Type	Required / default	Constraints & notes
`user`	→ users	required	indexed — the recipient
`type`	String	required	enum: `message · study-group · marketplace · system · friend` — drives the bell-icon grouping

`title` / `description`	String	required	what the bell shows
`read`	Boolean	`false`	unread badge count = `count({ user, read:false })`
`relatedId`	ObjectId	—	**polymorphic** via `refPath: "relatedModel"` — tap-through target
`relatedModel`	String	—	enum: `Listing · StudyGroup · Message · User · Conversation`
Indexes: `{ user, createdAt: -1 }` · `{ user, read }`			

19 · `devicetokens` models/DeviceToken.js

Field	Type	Required / default	Constraints & notes
`user`	→ users	required	indexed — one row per device per user
`university`	→ universities	required	indexed
`token`	String	required	**unique** — the FCM registration token; dead tokens are pruned automatically after failed sends
`platform`	String	`web`	enum: `web · android · ios`

20 · `friendships` models/Friendship.js

Field	Type	Required / default	Constraints & notes
`requester` / `recipient`	→ users	required	both indexed — who asked, who was asked
`users`	[→ users]	—	validated to exactly 2 · pre-validate hook sorts them (mirrors the Conversation idiom)
`pairKey`	String	—	**unique** — canonical `"<lowId>_<highId>"`. Pair-uniqueness lives on this scalar because a unique index on the `users` ARRAY would be multikey (= one friendship per user, total)
`status`	String	`pending`	enum: `pending · accepted` · indexed
`university`	→ universities	required	indexed
Indexes: `pairKey` unique · `{ users, status }`			

21 · `reports` models/Report.js

Field	Type	Required / default	Constraints & notes
`reporter`	→ users	required	indexed
`university`	→ universities	required	indexed
`targetType`	String	required	enum: `listing · document · confession · question · answer · lostfound · ride · event`

`targetId`	ObjectId	required	indexed — polymorphic, resolved through `targetType` (no `ref`)
`reason`	String	required	max 300
`snapshot`	{ title, content ≤500 }	—	a small **copy of the reported content** so the admin queue stays reviewable even after the target is deleted
`status`	String	`open`	enum: `open · resolved · dismissed` · indexed
`handledBy` / `handledAt`	→ users / Date	—	which admin closed it, when

Indexes: `{ university, status, createdAt: -1 }` · `{ reporter, targetType, targetId }` **unique** — one report per user per target (no spam-reporting)

> **Q: What's deliberately the same across all 21 collections?**
> A: Four conventions. (1) **Every content row carries** `university` — tenant isolation is a schema property, not an afterthought. (2) **Soft delete via** `isActive` — nothing user-facing is hard-deleted, so moderation and "undo" stay possible. (3) **Denormalized counters** (`upvoteCount`, `reactionsCount`, `answersCount`, `lastMessage`) wherever a list view sorts or previews. (4) **Timestamps everywhere** for free ordering and auditing.

8 · Data layer (continued) — where every piece of data travels

This chapter answers one question end-to-end: **when a user does something, which data leaves the device, what path does it take, and where does it finally come to rest?** Read it once and you can trace any byte in the system.

8-F.1 The only five places data can live

Store	What lives there	Lifetime
MongoDB Atlas	All 21 collections — including every uploaded *image* (as base64 data URLs inside documents)	Permanent. The single source of truth.
Cloudinary + its CDN	Study Vault file binaries (PDFs, docs, zips…). Mongo keeps only the `secure_url`	Permanent until the document is deleted (`_removeFile` / destroy)
The user's device	JWT + cached user JSON — in `localStorage` (sync mirror) AND Capacitor `Preferences` (durable native store)	Until logout or token expiry (~1 year)
API-server RAM	Presence map (`userId → Set<socketId>`), rate-limit counters, the cached Mongoose connection	Until restart/redeploy — all three are rebuildable, so losing them is harmless
In transit only	OTP / reset emails (SMTP → inbox), push payloads (firebase-admin → Google FCM → device tray)	Transient — the server keeps only hashes/flags, never the raw payloads

8-F.2 The master map — what goes where

Data	Leaves from	Travels via	Finally rests in
Password	Signup / login form	HTTPS JSON body	`users.password` as a bcrypt hash — plaintext is never stored and never returned (`select:false`)
JWT (the session)	Issued by the API on login/signup	Response body → device	Device only: `localStorage` + Capacitor Preferences. Never in the DB — comes back on every request as `Authorization: Bearer` and in the socket handshake
6-digit OTP / reset token	Generated server-side (`crypto`)	nodemailer → SMTP → student's inbox	Only its SHA-256 hash, in `users.emailVerificationToken` / `passwordResetToken` (30-min expiry). The raw code exists only in the email
Profile fields (name, bio, phone, privacy…)	Settings forms	`PUT /api/users/…`	`users` document
Avatar photo	Camera / gallery	Client-side resize → multipart → Multer (memory)	`users.avatarUrl` as an inline base64 data URL
Listing / Lost-&-Found photos, UPI QR, group covers	Gallery	Multipart → `dataUrlStorage`	Inline base64 in `listings.images[]` , `lostfounditems.images[]` , `users.paymentInfo.upiQrUrl` , `studygroups.image`

Study Vault file (PDF/doc/zip)	File picker	Multipart → Multer → **streamed** to Cloudinary (never buffered on the server)	Binary on Cloudinary/CDN; `documents` keeps only URL + metadata
Chat message text	Chat input	Socket.IO `sendMessage` event (or `POST /api/messages/:conversationId`)	`messages`; a preview copy is denormalized into `conversations.lastMessage`
FCM device token	Firebase SDK on the device	`POST /api/push/register`	`devicetokens` (unique per token; dead ones auto-pruned)
Push notification	Server (`utils/push.js`)	firebase-admin → Google FCM	Transient — ends in the device's notification tray. The in-app copy persists as a `notifications` doc
Confession	Compose box	`POST /api/confessions`	`confessions` — `author` is saved for moderation but never serialized to any client
Points & badges	Side-effect of contributions	`awardPoints()` — `$inc` / `$addToSet`	`users.points` / `users.badges`; the leaderboard just sorts on the indexed `points`
CGPA & attendance numbers	The student's own input	REST	`academicrecords` / `attendancesubjects` — per-user, never shown to anyone else
Report of abusive content	Report dialog	`POST /api/reports`	`reports`, with a `snapshot` copy of the offending title/content that survives deletion of the target
Online/offline presence	Socket connect / disconnect	—	Nowhere durable — an in-memory `Map` on the server, gone on restart, by design

> **The privacy invariant:** four things never leave the server in raw form — password hashes, confession/comment author ids, verification/reset token hashes, and `tokenVersion`. Everything a client receives passes through explicit field selection (`select:false` + hand-built response objects), so "anonymous" and "private" are enforced at serialization time, not by hoping the UI hides it.

8-F.3 Flow 1 — Signup & email verification (OTP)

```
Browser form { email, password, displayName }
  |  POST /api/auth/signup                    (HTTPS JSON)
  ▼
Express: authLimiter (20/15min) → scholar-email regex → resolveUniversityByEmail(domains)
  |  bcrypt pre-save hook hashes the password
  ▼
users doc { emailVerified:false,
            emailVerificationToken: sha256(6-digit OTP),  expires: +30 min }
  |  nodemailer → SMTP → student inbox  (the RAW code travels only here)
  ▼
POST /api/auth/verify-email { email, code }
  → sha256(code) compared to the stored hash → emailVerified:true
  → JWT issued → the user is in
```

If SMTP env vars are absent (local dev), signup skips verification entirely and returns the JWT immediately — graceful degradation, invariant #4.

8-F.4 Flow 2 — Login & where the session lives

```
{ email, password } → POST /api/auth/login
  → users.findOne(email).select("+password") → bcrypt.compare
  → JWT signed { id, university, tv: tokenVersion }   expires 365d
  ▼ response
Device:  localStorage ("token","user")     ← synchronous mirror for axios + socket
         Capacitor Preferences (same keys) ← durable native store (survives Android killing the WebView)
  ▼ every later call
REST:    Authorization: Bearer <jwt>       (axios request interceptor)
Socket:  io(url, { auth: { token } })      (checked in io.use before any event)
```

The server keeps **no session state**. Revocation is the `tv` claim: a password reset does `$inc tokenVersion`, and `middleware/auth.js` rejects any JWT whose `tv` no longer matches — every old token dies at once.

8-F.5 Flow 3 — the middleware gauntlet every request runs

```
request → helmet (headers) → CORS allow-list → express.json
        → apiLimiter (600/15min/IP) → connectDB() (cached connection)
        → route → auth (verify JWT, load user, reject banned/inactive, check tv)
        → universityScope (forces  { university: req.user.university }  into the query)
        → controller → { success, message, data }  →  errorHandler (if anything threw)
```

8-F.6 Flow 4 — a Study Vault upload (the only bytes that go to Cloudinary)

```
file picker → multipart/form-data POST /api/documents
  → Multer fileFilter  (pdf / doc(x) / ppt(x) / xls(x) / txt / png / jpg / webp / zip)
  → 20 MB cap
  → cloudinaryStorage: file.stream.pipe(upload_stream)   ← bytes stream through, never buffered
  → Cloudinary stores the binary, returns { secure_url, public_id, bytes }
  → documents doc { fileUrl, fileName, fileFormat, fileSize, uploader, university }
  → awardPoints(uploader, "document_upload")   (+15)

Later download: client opens fileUrl directly (Cloudinary CDN edge) —
our API only sees POST /:id/download, which $inc's downloadCount.
```

8-F.7 Flow 5 — every image upload goes straight into MongoDB

```
gallery/camera → client-side resizeImage() (shrinks before upload)
  → multipart POST → multer (memoryStorage | dataUrlStorage)
  → "…"  built in memory
  → saved INLINE in the parent document:
       users.avatarUrl · listings.images[] · lostfounditems.images[]
       users.paymentInfo.upiQrUrl · studygroups.image
  → rendering is just <img src={dataUrl}> — zero extra requests, zero external deps
```

> **Why inline data URLs and not Cloudinary, for images?** Production has no Cloudinary credentials — and images stored inline in Mongo survive redeploys with zero external services. The cost is document size, which is contained three ways: the client resizes before upload, Multer enforces tight `fileSize` caps, and images stay far below Mongo's 16 MB per-document ceiling. Documents (PDFs etc.) can't take this route — they're too big — which is exactly why the Study Vault pipeline is the one that keeps Cloudinary.

8-F.8 Flow 6 — a chat message, sender to receiver

```
sender types → socket.emit("sendMessage", { conversationId, content })
  → server: socket.user was set from the JWT at handshake → membership check
  → messages.create { conversation, sender, university, content }
  → conversations.$set { lastMessage, lastMessageAt }      (inbox preview, denormalized)
  → io.to(conversationRoom).emit("newMessage", …)          → receiver's screen updates live
```

```
  → receiver offline?  notifications doc  +  FCM push via their devicetokens

REST fallback:  POST /api/messages/:conversationId does the same writes,
                then emits through req.app.get("io") — one behavior, two transports.
Read receipt:   markConversationRead → messages.readAt = now → "seen" ticks.
```

8-F.9 Flow 7 — push, from permission to tray

```
device grants notification permission → Firebase SDK issues an FCM token
  → POST /api/push/register { token, platform } → devicetokens (unique)
  → any notification-worthy event calls sendPushToUser(userId, title, body)
  → firebase-admin sendEachForMulticast(tokens…) → Google FCM → device tray
  → FCM replies "token not registered"? → that token is deleted from devicetokens
```

8-F.10 Flow 8 — the three cron jobs (all run every 10 minutes)

```
node-cron "*/10 * * * *"
 ├─ classReminder:      timetableentries where dayOfWeek = today & startTime is near
 │                        → notify + push → lastReminderDay = "YYYY-MM-DD"  (idempotent)
 ├─ eventReminder:      events starting within ~1 h, reminderSent:false
 │                        → notify all attendees → reminderSent:true
 └─ studyGroupReminder: studygroups with nextSession.at near, sessionReminderSent:false
                          → notify all members → flag true (reset when a new session is set)
```

Each job writes its own idempotency flag back to the document — that flag *is* the reason reminders never fire twice even though the job wakes every 10 minutes.

8-F.11 Flow 9 — moderation, from post to ban

```
content posted → utils/contentFilter (text) + imageModeration → allow / block at the door
user hits "Report" → reports doc (with snapshot copy)   [confessions: embedded reports[]]
  → threshold crossed → isHidden:true + $inc author strikes
admin (isAdmin) → GET /api/reports → resolve / dismiss → handledBy + handledAt
strikes pile up / manual ban → users.isBanned:true
  → middleware/auth.js and the socket handshake both reject the account everywhere
```

> **Q: Trace "the seller's photo of a cycle" and "the buyer's ₹1,200 offer" end to end.**
> A: **Photo:** gallery → client resize → multipart POST `/api/listings/:id/images` → `dataUrlStorage` turns the bytes into a base64 data URL → pushed into `listings.images[]` — the image now lives inside the listing document in Atlas, and every viewer gets it inline with the listing JSON. **Offer:** form → POST `/api/offers` → controller loads the listing, denormalizes its `seller`, writes an `offers` doc (`status:"pending"`) → a `notifications` doc + FCM push go to the seller → the seller's accept/counter mutates `offers.status` — the negotiation is pure state-machine transitions on that one document.

9 · File & media storage — "how are the PDFs stored?"

This is the question you specifically asked about ("a library that breaks files into chunks, stores them, then rebuilds them"). That description is **GridFS** — and Manit Hub deliberately does **not** use it. Here is what it actually does, and why.

9.1 What actually happens when you upload a PDF (Study Vault)

For **Study Vault documents** the flow uses **Multer** (to parse the multipart upload) + a **custom streaming storage engine** that pipes the bytes straight to **Cloudinary** (cloud media host + CDN). The file's *bytes never touch the API server's disk or database* — only the resulting URL is stored in MongoDB. **Images are different:** avatars, listing photos, lost-&-found photos, UPI QR codes and group covers skip Cloudinary entirely and are stored *inline in MongoDB as base64 data URLs* — see §9.5 for the two engines and the why.

```
// middleware/cloudinaryStorage.js — a custom Multer StorageEngine
_handleFile: async (req, file, cb) => {
  const params = await paramsFactory(req, file);              // folder, public_id, resource_type
  const upload = cloudinary.uploader.upload_stream(params, (err, result) => {
    if (err) return cb(err);
    cb(null, { path: result.secure_url,    // ← the CDN URL
               filename: result.public_id, // ← handle for later deletion
               size: result.bytes });
  });
  file.stream.pipe(upload);   // STREAM bytes client → Cloudinary, never buffering to disk
}
```

```
// middleware/uploadDocument.js — wire it into Multer with limits + a type allow-list
const upload = multer({
  storage,                       // the Cloudinary streaming engine above
  fileFilter,                    // allow pdf/doc(x)/ppt(x)/xls(x)/txt/png/jpg/webp/zip only
  limits: { fileSize: 20 * 1024 * 1024 },   // 20 MB cap
});
```

Then the endpoint stores only metadata:

```
// documents/createDocument.js
const document = await Document.create({
   ... ,
  fileUrl: req.file.path,      // Cloudinary secure_url
  fileName: originalName,
  fileFormat: extension,       // derived from the original filename
  fileSize: req.file.size,
  uploader: req.user._id,
  university: req.user.university,
});
await awardPoints(req.user._id, "document_upload");  // +15 karma
```

9.2 So the real answer, in interview form

> **"PDFs are not chunked into the database. They are streamed through the API straight to Cloudinary, which stores the binary and serves it from a CDN. MongoDB only holds the file's URL plus metadata (name, format, size, uploader). Downloads/previews hit Cloudinary's URL directly, not our server."**

9.3 Why streaming (and not buffering)?

`file.stream.pipe(upload)` means a 20 MB PDF flows through in small buffers — the API process never holds the whole file in RAM, and nothing is written to the server's (often read-only / ephemeral) disk. This keeps the API **stateless and memory-flat**, which is exactly what a horizontally-scaled / serverless-ish host needs.

9.4 Why NOT the alternatives (the "why not X" the question implies)

> **Why not GridFS (the "chunk & rebuild" approach)?** GridFS splits a file into 255 KB chunks stored as documents in MongoDB and reassembles them on read. It exists for files larger than the 16 MB BSON document limit. But: (1) it bloats your database with binary data, inflating backups and storage cost; (2) **every download streams through your API server**, burning its bandwidth and CPU; (3) you get **no CDN, no image/PDF transforms, no global edge caching**. Cloudinary gives all of that for free and keeps the binary out of the DB. GridFS makes sense only if you're forbidden from using external storage.

> **Why not store the file as base64 in a Mongo field?** base64 inflates size by ~33%, you'd hit the 16 MB document limit fast, and you'd ship the whole blob through the API on every read. It's the worst of all worlds for anything but tiny thumbnails.

> **Why not local disk on the API server?** Hosts like Render/Vercel have ephemeral or read-only filesystems — files written on one deploy/instance vanish on the next or aren't visible to other instances. Disk storage breaks the moment you have more than one server. (The repo still mounts a legacy `uploads/` folder via `express.static` so very old avatar URLs keep resolving, but **no upload pipeline writes to disk anymore** — documents stream to Cloudinary, and every image type is stored inline in MongoDB as a data URL.)

> **Why not S3 directly?** S3 is a fine choice, but you'd add your own CDN (CloudFront), your own image/PDF transformation pipeline, and signed-URL plumbing. Cloudinary bundles storage + CDN + on-the-fly transforms behind one SDK — less to operate for a small team.

9.5 The upload pipelines — two storage engines, six middlewares

There are exactly **two** custom Multer storage engines, both exposing the result on `file.path` so route handlers don't care which one ran (a clean drop-in-replacement design):

- `cloudinaryStorage` — streams bytes to Cloudinary, `file.path` = the CDN `secure_url`. Used **only** for Study Vault documents.
- `dataUrlStorage` — buffers the bytes in memory and sets `file.path` = `"data:<mime>;base64,…"`, which the route saves **inline in the MongoDB document**. Used for every image type, because production has no Cloudinary credentials — inline images survive redeploys with zero external services. Kept safe by client-side resizing + tight `fileSize` caps (well under Mongo's 16 MB per-document limit).

Middleware	Used by	Engine → where the bytes end up
`uploadDocument`	Study Vault	**Cloudinary** — folder `manit-hub/documents`, `resource_type:auto`, 20 MB cap, pdf/office/text/image/zip allow-list → `documents.fileUrl`
`uploadAvatar`	Profile picture	Multer **memoryStorage** → route builds a base64 data URL → `users.avatarUrl`; 5 MB pre-resize cap, image-only, client resizes first
`uploadListingImages`	Marketplace photos	**dataUrlStorage** → `listings.images[]` (multiple per listing)
`uploadLostFoundImages`	Lost & Found photos	**dataUrlStorage** → `lostfounditems.images[]`

`uploadPaymentQr`	Seller's UPI QR	**dataUrlStorage** → `users.paymentInfo.upiQrUrl`
`uploadStudyGroupCover`	Group cover image	**dataUrlStorage** → `studygroups.image`

The Cloudinary engine also implements `_removeFile` (calls `cloudinary.uploader.destroy(public_id)`) so Multer can roll back the upload if the request later fails validation — no orphaned files. The data-URL engine needs no rollback: if the request fails, nothing was stored anywhere.

> **Q: How does a user *download* a document later?**
> A: The client just opens `document.fileUrl` (the Cloudinary CDN URL) — the bytes come from Cloudinary's edge, not from our API. The only thing our API does is `POST /documents/:id/download` to `$inc` a `downloadCount` for analytics/sorting. In-app PDF/image preview is done by pointing a viewer/iframe at that same URL.

> **Q: What stops someone uploading a 2 GB malicious EXE?**
> A: Two gates in the Multer config: a `fileFilter` with a MIME-type **allow-list** (only pdf/office/text/image/zip) and a `limits.fileSize` of 20 MB. The filter rejects disallowed types before any bytes are stored. (A further hardening would be server-side content sniffing, since MIME type from the client is advisory.)

10 · Real-time chat & presence — Socket.IO deep dive

Chat is the most stateful part of the system. It uses Socket.IO **rooms** for fan-out, the same JWT for auth, an in-memory registry for online presence, and carefully ordered handler registration to avoid a class of "dropped event" bugs.

10.1 Server bootstrap — `server.js`

```
const server = http.createServer(app);          // ONE http server…
const io = new Server(server, { cors:{ origin:"*" } });  // …shared by REST + Socket.IO
app.set("io", io);   // expose io so REST handlers can also emit (req.app.get("io"))
chatSocket(io);      // attach all socket logic
```

REST and WebSocket live on the same port/server. Exposing `io` via `app.set` lets a REST endpoint (e.g. a future "send message via HTTP") fan out in real time too.

10.2 Handshake auth (`io.use`)

```
io.use(async (socket, next) => {
  const token = socket.handshake.auth?.token;     // client sends it in io(url,{auth:{token}})
  const decoded = jwt.verify(token, JWT_SECRET);
  socket.user = await User.findById(decoded.id);
  if (!socket.user) return next(new Error("Unauthorized"));
  next();
});
```

10.3 Rooms & the events

Client emits	Server does	Server broadcasts
`join_conversation(id)`	verify participant + same university → `socket.join(id)`	—
`send_message{id,content}`	re-verify, `Message.create`, update conversation's `lastMessage`	`receive_message` to room `id`
`mark_read{id}`	set `readAt` on the other party's unread messages	`messages_read` to room

Each conversation is a **room** (named by its id). Emitting to `io.to(conversationId)` reaches exactly the two participants who have joined it — that's the fan-out primitive. Each user also joins a **personal room** `user:<uid>` used for presence pings.

10.4 The subtle bug that shaped the code: register handlers *synchronously*

> **Insight (literally commented in the code):** all `socket.on( ... )` handlers must be attached **before any** `await` in the connection callback. Socket.IO does **not** buffer events for handlers that aren't attached yet. The code originally did async DB work (loading presence) first; a client that emitted `join_conversation` immediately after connecting had it silently dropped — and then never received messages. Fix: register every handler first, then do the async presence announce afterward.

```
io.on("connection", async (socket) => {
  socket.join(`user:${uid}`);
  presence.addSocket(uid, socket.id);
  socket.on("join_conversation",  ... );   // ← all handlers registered FIRST,
```

```
  socket.on("send_message", ...);        //   synchronously, no await before them
  socket.on("mark_read", ...);
  socket.on("disconnect", ...);
  // …only NOW do async presence work (await User.find… etc.)
});
```

10.5 Message-notification coalescing

When a message is sent, the recipient gets a notification — but spamming 50 messages must not create 50 bell items. The code **upserts one unread notification per conversation**:

```
Notification.findOneAndUpdate(
  { user: rid, type:"message", relatedId: conversationId, read:false },  // match existing unread
  { ...fields, description: snippet },                                   // update its preview
  { upsert: true, setDefaultsOnInsert: true });                          // or create one
```

So a conversation shows as a single "New message from X" until the user reads it.

10.6 Presence — in-memory registry (`socket/presence.js`)

A module-level `Map<userId, Set<socketId>>`. A user is "online" while they hold ≥1 socket (multiple tabs/devices = multiple socket ids). `addSocket` returns "became online" only on the *first* socket; `removeSocket` returns "became offline" only on the *last* — so friend online/offline pings fire exactly once, not per tab. Presence also respects `privacySettings.showOnlineStatus` (opt-out users are simply omitted).

> **Why in-memory and not Redis?** It's the simplest thing that works on a single Node process and degrades gracefully: on a serverless/multi-instance host the map isn't shared, so everyone just reads as "offline" — no crash, just a missing nicety. The **scaling fix** (interview gold): introduce the Socket.IO Redis adapter so rooms and presence work across multiple server instances. That's the first thing to add when you go beyond one box.

10.7 The client side — a singleton socket service (`utils/socket.js`)

- **Singleton:** one connection shared app-wide (presence) and by the Messages page. `connect()` is idempotent.
- **Re-join on reconnect:** it keeps a `Set` of open conversation rooms and re-emits `join_conversation` for each on every `connect` event — because each reconnection is a fresh server-side socket that has joined nothing. Without this, messages silently stop arriving after a network blip.
- **Token in handshake:** `io(URL, { auth: { token: localStorage.token } })`.

> **Q: Why are messages sent over the socket but *fetched* over REST?**
> A: Loading history is a one-shot request/response (paged by `{conversation, createdAt}` index) — REST fits. Sending/receiving is the live, bidirectional part — that's the socket's job. `GET /messages/:id` reads history; `send_message` over the socket delivers new ones in real time. `PUT /messages/:id/read` and the `mark_read` socket event both exist so read state works whether or not the socket is connected.

> **Q: How are read receipts implemented?**
> A: Each Message has a nullable `readAt`. `mark_read` sets `readAt = now` on all messages in the conversation that the *other* person sent and that are still unread, then broadcasts `messages_read` so the sender's UI can show the ticks. Unread *counts* for the bell come from notifications, coalesced per conversation (10.5).

11 · Notifications & push

Two layers: an **in-app** notification (a Mongo document that powers the bell) and an optional **FCM push** that mirrors it to the OS/browser. The push layer is a *progressive enhancement* — if Firebase isn't configured, in-app notifications still work.

11.1 The single choke-point — `createNotification()`

```
// utils/notifications.js — used by ~every feature
async function createNotification(userId, type, title, description, relatedId, relatedModel) {
  const notification = await Notification.create({ user:userId, type, title, description, relatedId, relat
  sendPushToUser(userId, title, description);   // mirror to FCM (no-op if unconfigured)
  return notification;
}
```

Every feature (offers, rides, study-group joins, class reminders…) calls this one helper. `relatedId` / `relatedModel` are a **polymorphic reference** so a notification can point at a Listing, Conversation, Event, etc., and the UI can deep-link to it.

11.2 Push fan-out & dead-token pruning — `utils/push.js`

```
const devices = await DeviceToken.find({ user: userId });   // a user can have many devices
const response = await messaging.sendEachForMulticast({
  tokens: devices.map(d => d.token),
  notification: { title, body },
  data: { url: "/notifications" },
});
// FCM tells us which tokens are dead → delete them so we stop trying.
const dead = response.responses.filter(r => /not-registered|invalid-argument/.test(r.error?.code))…
await DeviceToken.deleteMany({ token: { $in: deadTokens } });
```

> **Self-healing token store:** phones uninstall, browsers revoke. Rather than let the token list rot, every send inspects FCM's per-token result and prunes the ones FCM reports as `registration-token-not-registered` / `invalid-argument`. The registry stays clean automatically.

11.3 Fire-and-forget discipline

Both `sendPushToUser` and `awardPoints` (gamification) are wrapped so they **never throw into the caller**. A failed push or a points hiccup must not break the action that triggered it (sending a message, posting an answer). They `try/catch` internally and log.

11.4 Registering a device — `POST /api/push/register`

The client obtains a token (web: Firebase SDK + service worker + VAPID; native: `@capacitor/push-notifications`) and posts it with a `platform` tag. Server upserts a `DeviceToken`. Both platforms hit the same endpoint — see Part 4.

Q: Web push vs native push — what's actually different?

A: The *token acquisition* differs. Web push needs a service worker (`firebase-messaging-sw.js`), a VAPID key, and the browser's Notification permission. Native Android push uses Google Play Services directly via the Capacitor plugin. But once a token exists, the **server treats them identically** — it just multicasts to every token a user has. That uniformity is the whole point of routing both through one endpoint and one `DeviceToken` model.

12 · Background jobs — node-cron reminders

Three periodic jobs run inside the API process: class reminders, event reminders, and study-group session reminders. They turn "you have a class at 10:00" into a push ~30 minutes before, with no user action.

12.1 How a job is structured (class reminder)

```
cron.schedule("*/10 * * * *", async () ⇒ {   // every 10 minutes
  const now = istNow();                       // campus-local time (IST), see below
  const entries = await TimetableEntry.find({
    dayOfWeek,
    startTime: { $gte: current, $lte: windowEndTime },  // starts in the next 30 min
    lastReminderDay: { $ne: today },                    // not already reminded today
  });
  for (const entry of entries) {
    await createNotification(entry.user, "system", `Upcoming class: ${entry.subject}`, …);
    entry.lastReminderDay = today;  await entry.save();  // mark as done → idempotent
  }
});
```

12.2 Two non-obvious correctness details

(a) Timezone independence. Hosts run in UTC; classes are in IST. The job computes an `istNow()` by adding the 5.5-hour offset, so "today" and "now" are always campus-local regardless of where the server lives.

(b) Idempotency via `lastReminderDay`. The job runs every 10 minutes, so a class in the 30-minute window would match several times. Stamping `lastReminderDay = today` and filtering `{ lastReminderDay: { $ne: today } }` guarantees **exactly one** reminder per class per day, even though the sweep is frequent.

12.3 Why node-cron (revisited)

Why in-process cron, not a queue (BullMQ/Agenda) or OS cron? The workload is a handful of light periodic *sweeps*, not millions of discrete delayed jobs. A queue would need Redis and a worker process; OS cron would need a server you manage and a separate entrypoint. node-cron is zero extra infra. **The catch (say this in interview):** if you run multiple API instances, each runs its own cron and you'd get duplicate reminders — so cron either runs on a single instance, or you'd move to a distributed scheduler / leader election. On a single Render web service it's perfect; it also won't fire on a purely serverless host that sleeps, which is one more reason the API runs as a long-lived Render service (see Part 17).

Q: A reminder cron that runs every 10 min — how do you avoid double-sending?
A: A per-entity "already done" marker. Here it's `lastReminderDay` on the timetable entry; the query excludes anything already stamped for today, and the entry is stamped immediately after sending. It's the same idempotency-key idea you'd use for webhooks or payment processing, scoped to a day.

13 · Feature-by-feature walkthrough

For each module: the endpoints (= functions), the libraries each uses, the workflow, and the interesting decisions. Every endpoint is `auth`-protected and `university`-scoped unless noted.

13.1 Marketplace (listings)

Endpoints: `getAllListings`, `getMyListings`, `getListingById`, `createListing`, `uploadListingImages`, `updateListing`, `updateListingStatus`, `deleteListing`.

Libraries: express, mongoose, multer + cloudinary (images). **Model:** Listing (enums for category/condition/status; text index).

The query builder (`getAllListings`) is the canonical "list endpoint":

```
const safeLimit = Math.min(Number(limit), 50);      // cap page size (defends against huge pulls)
const query = { isActive:true, university:req.user.university };
if (category && category≠="All") query.category = category;
if (minPrice||maxPrice) query.price = { ...$gte, ...$lte };
if (search) query.$or = [ {title:/…/i}, {description:/…/i} ];
const [listings, total] = await Promise.all([
  Listing.find(query).populate("seller","displayName avatarUrl").sort(sortOptions[sort]).skip(skip).limit(
  Listing.countDocuments(query),                    // run count in parallel for pagination meta
]);
```

> **Three reusable ideas here:** (1) page size is **capped server-side** (`Math.min(limit,50)`) so a client can't request 10,000 rows; (2) the data query and the count run **in parallel** with `Promise.all`; (3) sort is chosen from a **whitelist map**, never from raw user input (no injection into the sort spec).

> **Q: `$regex` search — what's wrong with it and what's the fix?**
> A: A leading-wildcard regex can't use an index, so it's a collection scan that gets slow at scale. The Listing/Document models actually declare a MongoDB **text index** (`title,description`); the production-grade fix is to use `$text` search (or Atlas Search) for relevance + index use. The regex is the pragmatic version that also matches substrings. Good thing to volunteer in an interview.

13.2 Offers — a negotiation state machine

Endpoints: `getOffers`, `createOffer`, `updateOffer` (accept/counter/decline/withdraw). **Helpers:** `populate.js`, `formatINR.js`. **Model:** Offer with `status: pending → countered → accepted | declined | withdrawn`.

createOffer workflow & guard rails: reject self-offers ("can't offer on your own listing"), reject if listing sold/inactive, enforce same-tenant, and block **duplicate active offers** from the same buyer (`findOne({listing,buyer,status:{$in:[pending,countered]}})`). On success it notifies the seller with a rupee-formatted amount.

```
const formatINR = (n) ⇒ `₹${Number(n).toLocaleString("en-IN")}`;  // ₹1,20,000 (Indian grouping)
```

> **Q: How do you model "negotiation"?**
> A: As a small **state machine** on the Offer document. Allowed transitions are enforced in `updateOffer`; an accepted offer can reserve the listing. Storing `seller` denormalized on the offer makes "offers I've received" a single indexed query without joining through the listing.

13.3 UPI checkout — client-side deep link + QR

Where: frontend `PayUpiModal.jsx` using the `qrcode` lib. **No server endpoint and no stored image** — the QR is a pure function of the seller's UPI ID + amount.

```
const buildUpiLink = ({upiId,name,amount,note}) ⇒
  `upi://pay?` + new URLSearchParams({ pa:upiId, pn:name, cu:"INR", am:amount, tn:note });
const qrDataUrl = await QRCode.toDataURL(upiLink, { width:280 });  // rendered as <img>
```

The seller's UPI ID comes from their `paymentInfo.upiId` (validated by a regex in the User schema). If they only uploaded a QR *image* (`upiQrUrl`, via the Cloudinary pipeline), the modal shows that instead. The button is an `<a href="upi://…">` that opens any UPI app.

> **Why generate the QR in the browser?** It's deterministic from inputs the client already has, so a server round-trip (and storing an image that's just a re-encoding of a string) is wasted work. The standard `upi://pay` deep link is what every Indian payment app understands — no payment-gateway integration, no money held by the platform ("payments go directly to the seller").

13.4 Study Vault (documents)

Endpoints: `getDocuments` (filter by branch/subject/semester/type + sort), `getMyDocuments`, `createDocument` (Cloudinary upload, Part 9), `incrementDownload`, `toggleUpvote`, `addComment`, `deleteDocument`. **Model:** Document with embedded `comments[]`, `upvotes[]` + denormalized `upvoteCount`.

- **toggleUpvote** adds/removes the user from `upvotes[]` and keeps `upvoteCount` in sync → powers the "Top rated" sort on an indexed scalar.
- **incrementDownload** is a cheap `$inc` for analytics; the file itself is fetched from Cloudinary, not the API.
- Uploading awards **+15 karma** + the "Note Sharer" badge.

13.5 Study Groups

Endpoints: `getAllStudyGroups`, `getUpcomingSessions`, `getStudyGroupById`, `createStudyGroup`, `updateStudyGroup`, `updateNextSession`, `uploadGroupCover`, `deleteStudyGroup`, `joinStudyGroup`, `leaveStudyGroup`.

Notable: member cap enforced on join; joining notifies the owner (respecting `notificationPreferences.studyGroups`); a cron job reminds members before a scheduled session. Mount order matters — `/upcoming` is registered before `/:id` so it isn't captured as an id.

13.6 Conversations & Messages

Endpoints: `createConversation`, `getUserConversations`, `getMessages`, `sendMessage` (REST companion), `markConversationRead`. Real-time delivery is the socket layer (Part 10).

Model design highlights (Conversation):

- **Exactly 2 participants** enforced by a schema validator.
- **Participants are sorted** in a `pre('save')` hook, and a **unique compound index** `{listingId, participants}` guarantees you can't create two conversations for the same pair about the same listing (idempotent "message seller").
- **Polymorphic context** (`contextType: listing | lostfound | ride | friend`) — one chat model serves marketplace, lost-&-found, rides, and friend DMs.
- `lastMessage` / `lastMessageAt` are denormalized so the conversation list renders + sorts without touching the Message collection.

Q: How do you prevent duplicate conversations between the same two people?

A: Sort the participant ids deterministically (`pre('save')`) and put a **unique index** on `{listingId, participants}` . Two attempts to start the same chat collide on the index; the create endpoint treats that as "return the existing one." Sorting is essential — without it `[A,B]` and `[B,A]` would look distinct to the index.

13 · Feature walkthrough (continued)

13.7 Confessions — provable anonymity

Endpoints: `getConfessions`, `createConfession`, `react`, `addComment`, `report`, `deleteConfession`. **Helper:** `serializeConfession.js` (uses Node `crypto`). **Model:** Confession (author stored, never serialized; embedded reactions/comments/reports; `isHidden`).

> **The anonymity guarantee:** the real `author` id is stored (so moderators can act on abuse) but is **never sent to clients**. Comment identities are deterministic hashes:
>
> ```
> anonHandle = "Anon-" + sha1(`${confessionId}:${userId}:manit-hub-anon`).slice(0,4).toUpperCase();
> ```
>
> This handle is **stable within a thread** (the same person is always "Anon-4F2A" in that confession), **different across confessions** (the confessionId is in the hash), and **not reversible** to the user. The original poster is shown as `"OP"`. The serializer also returns `isMine`, reaction counts, and `myReaction` without leaking who reacted.

> **Q: Why hash with both the confessionId and userId, plus a salt string?**
> A: Including `userId` makes the handle consistent for that user in the thread; including `confessionId` stops cross-thread correlation (you can't link "Anon-4F2A here" to "Anon-4F2A there"); the constant salt makes it non-trivial to brute-force a small id space offline. It's deterministic (no storage needed) yet privacy-preserving.

Moderation: `report` appends to `reports[]`; once reports cross a threshold the confession `isHidden` auto-flips (auto-hide), and admins can act via the Reports queue.

13.8 CGPA tracker

Endpoints: `getRecords`, `upsertSemester`, `deleteSemester`. **Model:** AcademicRecord (per-semester subjects/credits/grades). **Compute is client-side** (`components/cgpa/`): live SGPA per semester, cumulative CGPA, an SGPA trend chart, and a target-CGPA "what-if" (what SGPA do I need next sem to hit target X).

> **Why compute SGPA/CGPA on the client?** It's instant feedback while typing grades, needs no round-trip, and the server only persists the raw inputs. The math is a weighted average — cheap and better UX live. The server stays the source of truth for the *data*, not the derived numbers.

13.9 Attendance tracker — the "can I skip?" math

Endpoints: `getSubjects`, `createSubject`, `updateSubject` (present/absent/undo), `deleteSubject`; `ownedByUser` guard. **Client math (`attendanceMath.js`):**

```
// How many classes you can skip and stay ≥ target%
skippableClasses(attended, held, target) = max(0, floor(attended*100/target − held));
// How many in a row you must attend to climb back to target%
classesToRecover(attended, held, target) = ceil((target*held − 100*attended)/(100 − target));
```

> **These are closed-form solutions**, not loops. "Skip" solves `attended/(held+x) ≥ target` for x; "recover" solves `(attended+x)/(held+x) ≥ target` for x. Deriving them on a whiteboard is a perfect small algebra/edge-case interview question (note the guards: `held≤0`, `target≥100 → Infinity`).

13.10 Timetable

Endpoints: `getEntries`, `createEntry`, `updateEntry`, `deleteEntry` (+ `validateEntry`). Weekly grid by `dayOfWeek` + `startTime`; the class-reminder cron (Part 12) reads these. `lastReminderDay` lives here for idempotency.

13.11 Lost & Found

Endpoints: `getItems`, `createItem` (Cloudinary images), `updateStatus` (lost/found/returned), `deleteItem`. Posters are reachable via chat (Conversation `contextType:"lostfound"`).

13.12 Ride Share

Endpoints: `getRides`, `createRide`, `joinRide`, `leaveRide`, `deleteRide` (+ `populate.js`). Seat accounting on join/leave; cancelling a ride notifies booked passengers; coordinate via chat (`contextType:"ride"`). Posting awards karma + the "Road Tripper" badge.

13.13 Events & Clubs

Endpoints: `getEvents`, `createEvent`, `toggleRsvp`, `deleteEvent` (+ `serializeEvent.js`, `populate.js`). RSVP toggles attendance; the event-reminder cron pings attendees before start. Creating awards karma + "Event Host".

13.14 Course Q&A Forum

Endpoints: `getQuestions`, `getQuestion`, `createQuestion`, `upvoteQuestion`, `deleteQuestion`, `addAnswer`, `upvoteAnswer`, `acceptAnswer`, `deleteAnswer`. **Helper:** `helpers.js` (`withMyUpvote` strips the raw `upvotes[]` array and returns a boolean `myUpvote` instead — so the client learns "did *I* vote" without downloading the whole voter list).

Accepting an answer awards the answerer **+25 karma** + "Problem Solver". Models: Question, Answer (StackOverflow-style, tagged by branch/subject/semester).

> **Q: Why transform `upvotes[]` into `myUpvote` before sending?**
> A: Privacy + payload size. The client only needs the count and whether the current user voted; shipping every voter's id leaks who voted and grows unboundedly. `withMyUpvote` computes the boolean server-side and deletes the array from the response.

13.15 Friends & presence

Endpoints: `getFriends`, `getRequests`, `sendRequest`, `acceptRequest`, `removeRequest`, `unfriend` (+ `helpers.js`). **Model:** Friendship (`users[2]`, status). Accepted friends drive the live online/offline pings over the socket personal rooms (Part 10), gated by `privacySettings.showOnlineStatus`. Friend DMs reuse the Conversation model with `contextType:"friend"` and a null listing.

13.16 Reports & moderation

Endpoints: `createReport`, `getReports` (`isAdmin`), `handleReport` (`isAdmin`) (+ `targets.js` mapping report types to models). Any listing/document/confession/question can be reported (polymorphic target). Admins remove content or dismiss; duplicate reports on the same target auto-resolve together. This is the only place the `isAdmin` gate is used.

13.17 Leaderboard & gamification

Endpoint: `getLeaderboard` — top users by `points` (indexed) within the university, plus the caller's own rank. Points/badges are written by `utils/gamification.js` (Part 15).

13.18 Campus Maps

Where: frontend `CampusMaps.jsx` — a static, hand-curated list of ~37 MANIT locations (all 12 hostels with official Bhawan names, departments, canteens, sports grounds, landmarks), each with a Google Maps query (place name or exact lat/long). Search + category filter are client-side (`useMemo`); selecting a place swaps a Google Maps `output=embed` `<iframe>`.

> **Why a Google Maps iframe instead of Leaflet (which is in the deps)?** The iframe needs **no API key, no tile-server billing, no map data to host**, and it inherits Google's live POI data, satellite view, and "Directions" for free. Leaflet would give in-app interactivity but you'd supply tiles (Mapbox/OSM, often keyed/billed) and re-create POIs. For "show me where Hostel 6 is," the iframe is the lower-cost, higher-data-quality choice. The Leaflet deps remain for a future interactive iteration — worth stating honestly rather than claiming Leaflet is in use.

13.19 Dashboard & misc

Dashboard (`getDashboardSummary`): parallel `countDocuments` for active listings + active groups + a campus-locations constant, via `Promise.all` . **Universities** (`getUniversities`): list tenants. **Users**: ~16 endpoints — profile, avatar/QR upload, settings, saved-listings wishlist, handle availability check, search, account deletion, notification/privacy prefs.

14 · The frontend — React 19, routing, state, design system

14.1 App shell & provider tree (`main.jsx`)

```
<ThemeProvider>          // light/dark + native status-bar sync
  <BrowserRouter>        // client-side routing
    <AuthProvider>       // user + token (durable, see 14.3)
      <App/>             // routes + ToastProvider
</…>
```

Self-hosted `@fontsource` fonts are imported here so they're bundled (offline-ready in the WebView). Before the tree renders, `main.jsx` `await`s `bootstrapAuthStorage()` (14.3) — this hydrates the saved session out of durable native storage and into `localStorage` so the very first render already knows whether the user is signed in. The `await` is wrapped so render still happens if it fails (e.g. on the web, where there's nothing to hydrate).

14.2 Routing & code-splitting (`App.jsx`)

- Every screen is `React.lazy()` + a `<Suspense>` fallback → each route is its own JS chunk, so the initial load only ships the landing/auth code.
- Public routes (`/`, `/auth`) sit outside the guard; the rest are nested under `<ProtectedRoute>` → `<AppLayout>` (sidebar + topbar shell).
- `ProtectedRoute` simply checks `user` from context and `<Navigate to="/auth">` if absent — declarative auth gating.
- **The inverse guard:** the public landing (`/`) and auth (`/auth`) screens do the opposite — if a `user` is already present they `<Navigate to="/dashboard" replace>`. So a returning, signed-in user who opens the app at its start URL is taken straight into the app instead of the marketing page.
- Android hardware-back is wired here via `@capacitor/app`.

> **Q: What does lazy-loading routes buy you?**
> A: A smaller initial bundle and faster first paint — a student opening the login page doesn't download the marketplace, chat, and maps code. Vite/Rollup emits a separate chunk per `lazy()` import, fetched on demand when the route is visited. Combined with the manual vendor chunks (14.6), it keeps the critical path lean.

> **Q: Why does the landing page redirect signed-in users? Wasn't the session already saved?**
> A: The session *was* saved, but the app and the native shell both boot at the start URL `/`, which renders the marketing landing page. Without the redirect a returning user lands on a "Sign in / Get started" screen and *feels* logged out even though their token is intact — they'd sign in again needlessly. Redirecting `/` and `/auth` to `/dashboard` when `user` is set closes that gap. It's the routing half of "stay logged in"; the storage half (14.3) makes sure `user` actually survives an app restart in the first place.

14.3 Auth state (`AuthContext.jsx` + `utils/authStorage.js`)

Holds `user` in React state. `login()` / `logout()` delegate persistence to `authStorage.js` rather than touching storage directly. A nice defensive detail: it still **normalizes** `id` → `_id` (the API returns `user.id`, but the rest of the app expects Mongo's `_id`) and guards against corrupt/"undefined" stored values.

Two-tier storage (the "stay logged in" fix). `authStorage.js` keeps the session in two places and is the single writer for both:

- **Capacitor Preferences** — native `SharedPreferences` on Android — is the *durable* copy. It's what survives the OS killing the app process.

- `localStorage` is a *synchronous mirror*, because the axios request interceptor (14.4) and the Socket.IO handshake read the token synchronously and can't `await` a plugin call.

`persistAuth()` writes the `localStorage` mirror first (synchronously, so the token is usable immediately) then the durable copy; `clearAuth()` removes both. On boot, `bootstrapAuthStorage()` (called from `main.jsx` before render) treats Preferences as the source of truth and copies it into `localStorage`; if Preferences is empty but a legacy `localStorage` session exists, it migrates that into Preferences so an existing login isn't lost on the upgrade.

> **Q: Why wasn't plain `localStorage` enough — it persisted fine on the web?**
> A: On the web it is enough; `localStorage` survives a tab/browser close, so there the durable tier is a harmless no-op (web Preferences is itself `localStorage`-backed). The native Android WebView is the problem: it flushes `localStorage` to disk lazily and the OS wipes WebView storage when it reclaims the app's memory (swipe-close, low memory). That silently logged users out on almost every restart. Routing the durable copy through Preferences (native storage the OS doesn't clear) fixes it. Note this is only the *storage* half — the landing-page redirect (14.2) is what actually drops the returning user back into the app.

14.4 The axios layer (`api/axios.js` + 22 feature modules)

```
const api = axios.create({ baseURL: API_BASE_URL });
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;  // attach JWT once, globally
  return config;
});
```

One interceptor means **no endpoint ever manually sets the auth header**. The base URL resolves by environment: explicit `VITE_API_URL` → else dev uses `/api` (Vite proxy) → else the hosted backend. Each feature gets a thin module (`api/listings.js`, `api/offers.js`, …) of named functions, so screens import `listingsApi.getAll()` rather than sprinkling URLs.

14.5 The dev proxy (`vite.config.js`)

In dev, requests to `/api` are proxied to the backend (`changeOrigin:true`). The browser only ever sees same-origin `/api`, so **there are no CORS issues locally** — a common beginner pain point, solved by config.

14.6 Build optimization — manual vendor chunks

`vite.config.js` splits `node_modules` into `vendor-react`, `vendor-router`, `vendor-motion`, `vendor-socket`, `vendor-axios`, `vendor-icons`, `vendor-misc`. **Why:** these change rarely, so browsers cache them across deploys; only the app code chunk invalidates when you ship a feature.

14.7 The design system

- **Tailwind tokens** encode MANIT navy/crimson/gold, surfaces, and dark-mode variants (`tailwind.config.js`).
- **A `ui/` kit**: Button, Card, Modal, Input, Avatar, Badge, Skeleton, Spinner, Segmented, EmptyState, ConfirmModal, ToastProvider, ThemeToggle — composed everywhere for consistency.
- `cn()` (`clsx` + `tailwind-merge`) for safe conditional classes.
- **⌘K command palette** (`command/CommandPalette.jsx`) for fast navigation.
- **8-item hub navigation** (Academics · Study · Marketplace · Campus Life) with in-page tabs, responsive SideNav/BottomNav/MobileDrawer.

14.8 Theming (`ThemeContext.jsx`)

Initial theme = saved choice → else OS `prefers-color-scheme`. Toggling adds/removes `.dark` on `<html>` and persists to localStorage. On native it also recolors the Android status bar to match — a small touch that makes the

WebView feel native.

14.9 Hooks

`useAuth` (context accessor), `useUnreadCount` (polls unread notifications, re-fetching on every route change via `useLocation`), `usePresence` (subscribes to socket presence events). `useUnreadCount` is a pragmatic "poll on navigation" rather than a live socket counter — cheap and good enough for a bell badge.

> **Q: Why Context for auth/theme instead of Redux?**
> A: The shared global state is tiny — current user, token, theme, presence. React Context covers it with zero dependencies. Redux's boilerplate and middleware pay off when you have lots of cross-cutting, frequently-updated state with complex derivations; this app doesn't. Server data is fetched per-screen via the axios modules. If caching/refetching got complex I'd reach for React Query before Redux.

15 · Gamification — points, badges, leaderboard

One helper drives it all (`utils/gamification.js`): `awardPoints(userId, action)`.

Action	Karma		Milestone	Badge
answer_accepted	+25		100 pts	Rising Star
document_upload	+15		500 pts	Campus Hero
answer_posted / event_created	+10		1000 pts	MANIT Legend
question/listing/ride/lostfound	+5		+ action badges: Note Sharer, Problem Solver, Event Host, Road Tripper	
confession / comment	+2		awarded the first time the action happens	

How it works: `$inc` the user's `points`, then grant any action-badge and any milestone badge now crossed, deduped with `$addToSet`. It's **fire-and-forget** (wrapped in try/catch, never throws) so a points failure can't break the upload/answer that earned them. The leaderboard sorts on the indexed `points` field, scoped to the university.

> **Q: Why must `awardPoints` never throw?**
> A: It's a side-effect of a primary action. If awarding points failed and bubbled up, a user could upload a valid document and still get a 500. Decoupling "the thing happened" from "the reward was recorded" keeps the core action reliable; the worst case of a points failure is a missing badge, which is logged.

16 · Security model — threats & mitigations

Threat	Mitigation in the code
Password theft from a DB leak	bcrypt (cost 10, salted) via the model `pre('save')` hook; `password` is `select:false`
Token forgery	JWT signed with a server secret; the auth middleware also re-checks the token's `university` claim against the live user
Cross-tenant data access	Every query filtered by `req.user.university`; `universityScope` for `:id` routes; socket checks tenant before join/send
Email enumeration	Login + forgot-password return generic messages regardless of whether the email exists
Reset-link theft from a DB leak	Only the SHA-256 **hash** of the reset token is stored, with a 30-min expiry
Privilege escalation	Admin actions gated by `isAdmin` middleware that reads the server-loaded `req.user`, never the request body
Malicious uploads	Multer MIME allow-list + 20 MB size cap; files isolated on Cloudinary, off the app server
Privacy leaks (confessions)	Author id never serialized; identities are one-way SHA-1 handles
Common web headers	`helmet` sets secure response headers; CORS uses an explicit origin allow-list
NoSQL injection	Mongoose casts query values to the schema types; sort uses a whitelist map, not raw input

2026 hardening pass — most of the classic gaps are now closed. Rate limiting (`express-rate-limit`) is live on the API and auth routes; the password-reset email is wired via `nodemailer` (token only returned in dev when no SMTP is set); email verification proves ownership of the scholar address at signup; JWTs carry a `tokenVersion` so a password reset instantly revokes old sessions; uploads reject SVG and enforce a raster-only MIME allow-list; and a content-moderation layer (text filter + report auto-hide + strikes/bans + optional image moderation) now guards user-generated content. Full details in the continued section below.

Honest gaps that remain (still worth raising in an interview): JWT in `localStorage` is XSS-exposed (vs an httpOnly cookie) — a deliberate trade to keep the Capacitor Android client simple; MIME is validated by allow-list but not yet by magic-byte sniffing; input shape/length validation is still hand-rolled per endpoint rather than a schema library (`zod` / `express-validator`); and the rate-limiter uses an in-memory store, correct for the single Render instance but needing a shared store (Redis) if scaled horizontally.

16 · Security hardening & content moderation (continued)

This section documents the 2026 security pass in full: **what** each control is, **why** it's needed, **how** it's implemented in this codebase, and — the question that actually gets asked — **why this approach and not an alternative**. The philosophy throughout is **defence in depth**: no single control is trusted alone; each layer assumes the one in front of it might fail.

16.1 · Rate limiting — throttling abuse

Why it's used. Without a request cap, an attacker can brute-force passwords, spam signups, or guess password-reset/verification codes as fast as the network allows. Rate limiting turns an online guessing attack into an infeasible one.

How it's implemented. `express-rate-limit` with two tiers, plus `app.set("trust proxy", 1)` so the real client IP (not Render's proxy) is counted:

```
// broad cap on the whole API
apiLimiter  → 600 requests / 15 min / IP
// strict cap on auth, counting only FAILURES
authLimiter → 20 failed attempts / 15 min / IP  (skipSuccessfulRequests:true)
```

Because the auth limiter uses `skipSuccessfulRequests`, a legitimate user logging in normally is never throttled — only repeated *failures* (the signature of brute force) burn the quota.

> **Why this, not nginx/Cloudflare/fail2ban?** App-level limiting travels with the code — it works identically in local dev, on Render, and on any future host, with no infra to configure and full knowledge of *which* route and *which* outcome (success vs failure) is being limited. An edge/proxy limiter can't distinguish a failed login from a successful one, which is exactly the distinction that makes the auth limiter precise.

16.2 · Email verification & the scholar-email rule

Why it's used. The whole app's value is that it's a *verified, students-only* community. Anyone could otherwise register a scholar number they don't own and impersonate a real student.

How it's implemented. Two gates. First, a regex (`/^\d{10}@stu\.manit\.ac\.in$/`) enforced at the model, at signup, at login, and on profile-email change — so only a MANIT scholar address is even accepted. Second, when SMTP is configured, signup generates a **6-digit code**, stores only its **SHA-256 hash** with a 30-minute expiry, emails the plaintext, and blocks login until the code is confirmed. Proving you can read the code in that inbox proves you own the address.

> **Why a 6-digit code, not a magic link?** The client is also a Capacitor Android app; a tapped web link doesn't reliably deep-link back into the native app, whereas a code can be typed into whatever screen the user is already on. The code is hashed at rest (a DB leak reveals nothing usable) and short-lived, and the auth rate-limiter makes guessing the 10^6 space within 30 minutes infeasible.

16.3 · Token versioning — instant session revocation

Why it's used. Sessions are deliberately long-lived (≈1 year) so students aren't logged out constantly. But a long-lived stateless JWT can't normally be revoked — if a token is stolen, it's valid until it expires. That's unacceptable for the "I think someone got into my account" case.

How it's implemented. A `tokenVersion` integer on the user is embedded in every JWT as a `tv` claim. The auth middleware (and the socket handshake) reject any token whose `tv` ≠ the user's current version. A password reset

increments `tokenVersion`, which **instantly invalidates every previously issued token**. Legacy tokens with no `tv` claim are treated as version 0, so nobody is logged out by the upgrade itself.

> **Why not just short expiry + refresh tokens?** Refresh-token rotation is the "correct" enterprise answer but adds a token store, refresh endpoints, and client retry logic — a lot of surface for a student app. A single version integer gives the one property that actually matters here (revoke-on-compromise) for the cost of one DB field and one comparison, while keeping the long, friction-free session the product wants.

16.4 · CORS & the substring trap

Why it matters. CORS decides which web origins may call the API with credentials. A loose rule lets a hostile website make authenticated-looking cross-origin requests.

How it's implemented. An exact allow-list of the known web origins, plus preview deploys matched on the **full project slug as a subdomain label** — not a loose substring.

> **The bug this fixes.** An earlier rule allowed any host *containing* `"manithub"`. But anyone can register a free Vercel project named `manithub-evil.vercel.app` — which contains the substring and would have passed. The fix requires the slug to be the actual leftmost DNS label (`manithub-samayjainbm` exactly, or `slug-…` / `…--slug` for previews), which an attacker can't forge. Socket.IO's CORS (previously `origin:"*"`) now shares the same policy.

16.5 · Mass assignment — whitelisting writable fields

Why it's used. A handler that does `Object.assign(doc, req.body)` lets a client set *any* field on the document — including ones it should never control.

How it's implemented. Listing update now copies only an explicit whitelist (`title, description, price, category, images, condition, status`). The dangerous fields — `seller`, `university`, `isActive`, timestamps — can no longer be set from the body.

> **Why a whitelist, not a blacklist?** A blacklist of "fields to strip" silently fails open the day someone adds a new sensitive field and forgets to list it. A whitelist fails *closed*: a new field is non-writable by default until you deliberately allow it. For an ownership/tenant boundary, fail-closed is the only safe default — this is what stopped a seller from moving their listing into another campus's marketplace.

16.6 · Upload safety — the SVG problem

Why it's used. A naive `mimetype.startsWith("image/")` check also accepts `image/svg+xml`. An SVG is XML that can carry `<script>`, so if it's ever rendered inline it becomes stored XSS.

How it's implemented. A single shared `imageFileFilter` allows only raster types (`jpeg, png, webp, gif`) and is wired into all five image uploaders. Image counts are capped (≤6 per listing), and — separately — every profanity check and the image-moderation gate run **before** the file/text is persisted.

> **Why an allow-list of raster formats, not "block SVG"?** Same fail-closed logic as 16.5: enumerating the formats you accept is safer than trying to enumerate every dangerous format you must reject. Anything exotic (SVG, HTML masquerading as an image, a novel type) is denied by default.

16.7 · Content moderation — keeping explicit/abusive content out

Why it's used. A campus app with confessions, a marketplace, forums and image uploads is a target for pornographic and abusive content. Moderation has to stop the obvious cases automatically and give humans the tools to handle the rest.

How it's implemented — three layers:

Layer	What it does	Where
Text filter	Self-contained EN + Hindi/Hinglish profanity list that un-obfuscates leetspeak (`p0rn`), spacing (`f u c k`) and repeats (`fuuuck`), while whole-word matching avoids false positives (`class` , `assignment` stay clean). Rejects the post with a 400.	confessions, forum Q&A, listings
Auto-hide + strikes/bans	Content auto-hides after *N* distinct reports; the author gets a strike; enough strikes auto-suspends the account. A banned user is blocked at login, on every API call, and at the socket handshake.	reports + `User.isBanned/strikes`
Image moderation	Env-gated gate (Google Cloud Vision SafeSearch or Sightengine) that rejects adult/explicit imagery **before it is stored**. No-op until a provider key is set, so nothing breaks by default.	avatar + listing uploads

Why a self-contained text filter, not an AI moderation API for text? Text abuse is dominated by a known vocabulary, so a local list catches the vast majority at **zero latency and zero cost**, with no third-party dependency for every post. The tunable `EXTRA_BLOCKED_WORDS` env lets campus-specific slang be added without a code change. AI text moderation is a fine future upgrade but overkill for the 95% case.

Why is image moderation a paid API, but off by default? Reliable nudity detection genuinely needs a trained model — a keyword list can't "read" an image. Rather than ship a heavy on-server model that strains Render's free tier, the code integrates a cloud SafeSearch API behind an env flag: enable it with one key when you're ready, and until then the report-queue + strikes/bans still cover images. The gate **fails open** on a provider outage so a third-party blip can't block every upload — the human report queue remains the backstop.

Why *only* this stack (and not more)? The controls are chosen for a **student-run app on free-tier infra**: each one buys a large reduction in a real risk for very little cost or complexity, and they layer so no single failure is fatal (filter miss → report → auto-hide → strike → ban; moderation API down → fail open → human queue). Heavier machinery — refresh-token rotation, a WAF, on-prem ML moderation, a validation framework — is deliberately deferred: it would add cost and operational burden out of proportion to the threat this app faces today, and every one of them has a clear, documented upgrade path when the app grows.

17 · Deployment & infrastructure

Component	Host	Why
Web app	Netlify	Static SPA: `vite build` → publish `dist/`, with a redirect of all paths to `index.html` (client routing)
API + Socket.IO	Render	A **long-lived** Node web service — required for persistent WebSocket connections and node-cron
Database	MongoDB Atlas	Managed Mongo; host IPs whitelisted
Media	Cloudinary	Binary storage + CDN (Part 9)
Push	Firebase Cloud Messaging	Web + Android delivery
Android	Google Play	Signed `.aab` from the Capacitor project

Why Render and not Vercel for the API? This is the single most important infra decision. Vercel (and serverless generally) runs your code in **short-lived functions that can't hold a WebSocket open** and don't keep an in-process cron alive. Manit Hub's real-time chat, in-memory presence, and reminder jobs all need a **persistent process** — so the API runs as a long-lived Render web service. (The codebase still has serverless-safe touches — the cached `connectDB` promise, "presence degrades to offline" — from an earlier Vercel attempt; the git history literally ends with "remove vercel".)

Q: How does client-side routing survive a page refresh on a static host?

A: A catch-all redirect (`netlify.toml` / SPA fallback) rewrites every path to `index.html`, so React Router takes over on load. Without it, refreshing `/marketplace` would 404 because no such file exists on the static host.

18 · Performance & scaling — what breaks first, and the fix

Concern	Current design	The fix when it gets big
Large lists	Skip/limit pagination, page size capped at 50, parallel count	Cursor/keyset pagination (skip is O(n) deep in); cache hot pages
Search	`$regex` (works, but can't use an index with leading wildcard)	`$text` index (already declared) or Atlas Search
Top sorts	Denormalized counters (`upvoteCount` , `reactionsCount`) on indexed fields	Already optimized; add caching for the leaderboard
Auth DB hit per request	`findById` with a fixed projection	Cache user in Redis keyed by id; short TTL
Real-time across instances	In-memory presence + single-process Socket.IO	**Socket.IO Redis adapter** — share rooms/presence across N servers
Cron on multiple instances	In-process node-cron (fine on one box)	Run cron on one instance / leader election / external scheduler
Bundle size	Route-level lazy loading + manual vendor chunks + dynamic Firebase import	Already strong; add route prefetch on hover
Media bandwidth	Cloudinary CDN; binaries never touch the API	Add Cloudinary transforms (resize/format) per surface

> **The headline scaling story:** the REST layer is **stateless** (JWT + no local files + cached DB connection), so it scales horizontally trivially. The **only** stateful pieces are Socket.IO rooms and the in-memory presence map — and both have a known, standard fix (the Redis adapter). That's the cleanest possible answer to "how would you scale this?"

19 · Rapid-fire interview question bank

Cross-cutting questions an SDE-1 panel would actually ask about this codebase. Each answer is one or two sentences — expand from the relevant part above.

System design

> **Q: Walk me through what happens when a user sends a chat message.**
> Client emits `send_message` over the authed socket → server re-verifies participant + tenant → `Message.create` → updates the conversation's `lastMessage` → `io.to(room).emit('receive_message')` to both participants → upserts **one** coalesced unread notification for the recipient → which mirrors to FCM push.

> **Q: How is the app multi-tenant?**
> Shared DB, row-level isolation by a `university` ObjectId on every document; the tenant always comes from the server-trusted `req.user`, enforced on REST queries, `universityScope` middleware, and the socket layer.

> **Q: One codebase ships to web and Android — how?**
> Capacitor wraps the Vite `dist/` in a native WebView shell. Same JS; only tiny runtime branches keyed on `Capacitor.isNativePlatform()` (HTTP transport, push, status bar, back button).

Backend

> **Q: Why one file per endpoint?**
> Isolated diffs, route name = filename, each file is self-contained and readable end-to-end; shared logic lives in `utils/` or feature `helpers.js`.

> **Q: How do you avoid opening many DB connections under load?**
> Cache the connection *promise* in `connectDB` so concurrent first requests await one in-flight connect; reuse the live connection once `readyState===1`.

> **Q: How are files stored?**
> Multer streams uploads straight to Cloudinary; Mongo stores only the URL + metadata. Not GridFS, not base64, not local disk — see Part 9 for why each is rejected.

> **Q: How do you stop a reminder cron from double-firing?**
> A per-entity day marker (`lastReminderDay`) excluded in the query and stamped after send → exactly one reminder per item per day, despite a 10-min sweep.

Auth & security

> **Q: Why bcrypt and not SHA-256?**
> bcrypt is deliberately slow + salted to resist brute force; SHA-256 is fast and unsalted — wrong tool for passwords.

Q: Stateless JWT — how do you revoke/ban?

The auth middleware re-loads the user each request, so flipping `isActive` (or deleting the user) takes effect on the next call; 7-day expiry bounds stale tokens.

Q: What would you harden first?

Rate limiting on auth/writes, move JWT to an httpOnly cookie, wire the reset email, add schema validation (zod), and a tenant-injecting query helper.

Data modeling

Q: Embed vs reference — give an example of each here.

Embed: confession reactions/comments (always read with the parent, bounded). Reference: Message→Conversation (unbounded, paged independently).

Q: Why denormalize `upvoteCount`?

So "Top rated" sorts on an indexed scalar instead of computing array lengths across the collection; cost is keeping it in sync on each vote.

Q: How do you guarantee one conversation per pair per listing?

Sort participant ids in a pre-save hook + a unique compound index on `{listingId, participants}`.

Frontend

Q: How is the JWT attached to requests?

A single axios request interceptor reads it from localStorage and sets the `Authorization` header globally.

Q: How do you keep the initial bundle small?

Route-level `React.lazy` + Suspense, manual vendor chunks in Vite, and dynamically importing Firebase only when push is configured.

Q: No CORS pain in dev — why?

Vite proxies `/api` to the backend, so the browser only sees same-origin requests.

Algorithms (the two whiteboard-able ones)

Q: Derive "how many classes can I skip?"

Solve `attended/(held+x) ≥ target/100` for the largest integer x → `floor(attended·100/target − held)`, clamped at 0.

Q: Generate a stable but anonymous comment handle.

`SHA-1(confessionId : userId : salt)`, take 4 hex chars → consistent in-thread, uncorrelated across threads, irreversible.

---

before an interview.
Built by students, for students. 🎓

20 · Appendix — the complete function-by-function reference

Parts 1–19 explained the *ideas*. This appendix is the **exhaustive catalogue**: every backend endpoint and every shared function, then the key frontend functions. Read a card and you know what that function does, which libraries it pulls in, the exact order it does things, and why it's built that way. Use it as a lookup while studying — the cards are grouped exactly like the folders on disk.

How to read a card. The mono line is the **HTTP method + path** (or function name) and the **file** it lives in. Then:

- **Libs / Guards / Helpers** — the npm libraries it imports, the middleware chain that runs before it (`auth` → JWT, `scope` → `universityScope(Model)`, `isAdmin`), and the shared utils it calls.
- **Does** — one line of purpose. **Flow** — the numbered steps. **Why** — the decision/edge-case worth saying out loud in an interview.

Two conventions hold for almost every backend card, so they're not repeated: (1) the file is a one-route Express `Router` mounted in `app.js`; (2) the handler is wrapped in `try/catch` and forwards errors with `next(err)` to the global error handler. The standard reply is `success(res,data,msg,status)` / `error(res,msg,status)`.

20.1 Bootstrap & configuration

`server.js — process entry point`

Libs: http, socket.io, dotenv · **Imports:** app, connectDB, chatSocket, 3 cron jobs

Does: Boots the whole backend. **Flow:** 1) `dotenv.config()` loads `.env`; 2) wrap the Express `app` in a raw `http.createServer`; 3) attach a Socket.IO `Server` to that same HTTP server (`cors.origin:"*"`); 4) `app.set("io", io)` so REST handlers can emit too; 5) `chatSocket(io)` registers all socket logic; 6) `connectDB()`, then start the 3 reminder cron jobs, then `server.listen(PORT||5001)`. **Why:** one HTTP server carries both REST and WebSocket → one port, one deploy, one JWT scheme. Cron starts only here (the long-lived process), never on import.

`app.js — the Express application`

Libs: express, cors, helmet, path · **Imports:** errorHandler, connectDB, ~150 routers

Does: Builds the middleware pipeline and mounts every endpoint. **Flow:** helmet → dynamic CORS (allow-list + `*.netlify.app` / `*.vercel.app` + localhost + Capacitor schemes) → `express.json()` / `urlencoded` → static `/uploads` → a **DB-connection gate** (`await connectDB(); next()`) → all feature routers under `/api/*` → `/api/health` + `/` → **errorHandler last**. **Why:** order matters — security headers first, body parsed before handlers, DB guaranteed connected before any route, error handler dead last so every throw becomes JSON.

`connectDB() — config/db.js`

Libs: mongoose

Does: Returns a single shared Mongo connection. **Flow:** guard missing `MONGO_URI` → refuse a localhost URI in production → if `readyState===1` reuse the live connection → if a connect is in flight reuse that **promise** → else `mongoose.connect(uri,{serverSelectionTimeoutMS:30000})`, cache the promise, reset to `null` on failure. **Why:** caching the *promise* (not a boolean) stops a thundering herd — ten concurrent first requests await one connect instead of opening ten.

`cloudinary — config/cloudinary.js`

Libs: cloudinary v2

Does: Configures the Cloudinary SDK from `CLOUDINARY_*` env vars and exports the singleton used by every upload engine. **Why:** one place holds the credentials; the SDK instance is reused everywhere.

`{ messaging } — config/firebase.js`

Libs: firebase-admin

Does: Initialises Firebase Admin for FCM. **Flow:** read `FIREBASE_SERVICE_ACCOUNT` (raw JSON *or* base64) → `JSON.parse` → `initializeApp({credential:cert( … )})` → export `messaging`. If the env is unset or parsing fails, exports `messaging=null`. **Why:** push is a **progressive enhancement** — a missing/broken key disables push silently, never crashing boot; in-app notifications keep working.

20.2 Middleware

`auth — middleware/auth.js`

Libs: jsonwebtoken · **Model:** User

Does: Authenticates every protected request. **Flow:** read `Authorization: Bearer <jwt>` → `jwt.verify` → `User.findById(decoded.id)` with a **fixed lean projection** (id, email, displayName, handle, university, isAdmin, points, badges, savedListings…) → 401 if no user → **tamper check:** token's `university` claim must equal the live user's → attach `req.user` + `req.token`. **Why:** re-loading the user each request buys instant ban/revocation and fresh fields, for one indexed read.

`universityScope(Model, paramKey="id") — middleware/universityScope.js`

Libs: mongoose

Does: A factory that guards single-resource `:id` routes against cross-tenant access. **Flow:** validate the ObjectId (400) → `Model.findById` (404) → 500 if the doc has no `university` → compare `resource.university` vs `req.user.university` (403) → attach `req.resource` so the handler needn't re-fetch. **Why:** one reusable tenant gate instead of copy-pasting the same compare into every endpoint; returning `req.resource` also saves a query.

`isAdmin — middleware/isAdmin.js`

Does: Gates the moderation queue. **Flow:** 403 unless `req.user.isAdmin`. **Why:** must run *after* `auth` (it reads the server-loaded user, never the request body) — the only privilege check in the app.

`errorHandler(err,req,res,next) — middleware/error.js`

Does: The single global error sink (4-arg = Express error middleware). **Flow:** log → status = `err.statusCode||err.status||500` → `res.status(status).json({message})`. **Why:** mounted last so any thrown/`next(err)` becomes clean JSON, never an HTML stack trace leaking internals.

`createCloudinaryStorage(paramsFactory) — middleware/cloudinaryStorage.js`

Libs: cloudinary

Does: A **custom Multer StorageEngine** that streams uploads straight to Cloudinary. **Flow:** `_handleFile` → run the per-type `paramsFactory` (folder/public_id/resource_type) → open `cloudinary.uploader.upload_stream` → `file.stream.pipe(upload)` → return `{path:secure_url, filename:public_id, size:bytes}`; `_removeFile` calls `cloudinary.uploader.destroy(public_id)`. **Why:** streaming means the API never buffers the file to RAM or disk (stateless, memory-flat); `_removeFile` lets Multer roll back a failed request so no orphan files. This factory is reused by 5 of the 6 upload middlewares.

`uploadDocument — middleware/uploadDocument.js`

Libs: multer + the Cloudinary engine

Does: The Study-Vault upload middleware. **Config:** folder `manit-hub/documents`, `resource_type:auto`, public_id `document_<uid>_<ts>`; a `fileFilter` MIME **allow-list** (pdf/doc(x)/ppt(x)/xls(x)/txt/png/jpg/webp/zip) and a **20 MB** cap. **Why:** the filter rejects bad types *before* any byte is stored; `resource_type:auto` lets Cloudinary host PDFs and images alike.

`uploadListingImages · uploadLostFoundImages · uploadPaymentQr · uploadStudyGroupCover — middleware/upload*.js`

Libs: multer + the Cloudinary engine

Does: Four image-only variants of the same factory, each with its own Cloudinary folder, an **on-upload transform** (`crop:"limit"` + `quality:"auto"` — listings/lost-found 1200², group cover 1400×800, QR 600²) and a size cap (5–10 MB). **Why:** server-side resize/compress at upload time keeps stored images small and bandwidth low; one factory, six configs (the factory pattern in practice).

`uploadAvatar — middleware/uploadAvatar.js`

Libs: multer (diskStorage), fs, path

Does: The **only disk-based** upload — writes avatars to `uploads/avatars/avatar—<uid>—<ts>.ext`, image-only, 5 MB. **Why:** a deliberately simpler path for small profile pics served by `express.static`. **Honest caveat:** disk is ephemeral on cloud hosts, so unlike the Cloudinary uploads, avatars don't survive a redeploy — the first thing you'd migrate to Cloudinary too.

20.3 Utilities (`utils/`)

`success() · error() — utils/response.js`

Does: The response envelope. `success` → `{success:true, message, data}`; `error` → `{success:false, message, …(extra only when NODE_ENV≠production)}`. **Why:** one shape lets the frontend always read `res.data.data` and toast `res.data.message`; debug `extra` is hidden in prod.

`generateToken(userId, universityId) — utils/jwt.js`

Libs: jsonwebtoken

Does: Signs the auth token: `jwt.sign({id,university}, JWT_SECRET, {expiresIn:"7d"})` (HS256). **Why:** embedding `university` lets middleware tenant-check without a query; 7-day expiry bounds a stolen token's lifetime.

`handle helpers — utils/handle.js`

Does: `isValidHandle` (regex `^[a-z0-9_.]{3,20}$`), `sanitizeBase` (strip an email/name down to a legal base), and `generateUniqueHandle(User, seed)` — try the base, then `base1, base2…` until `User.exists({handle})` is false (timestamp fallback after 1000 tries). **Why:** guarantees a unique, URL-safe `@handle` at signup; `User` is passed in to dodge a circular require.

`resolveUniversityByEmail(email) — utils/universities.js`

Model: University

Does: Maps an email to its tenant. **Flow:** take the domain after `@` → `University.findOne({domains:domain})` → if none, **lazily create** one (`name` = first label upper-cased, `domains:[domain]`). **Why:** the email domain *is* the verification + the tenant key, so onboarding a new campus = just letting a new domain sign up.

`awardPoints(userId, action) — utils/gamification.js`

Model: User

Does: Grants karma + badges. **Flow:** look up the action's point value (POINTS map) → `$inc user.points` → add the action-badge (Note Sharer / Problem Solver / Event Host / Road Tripper) → add any milestone badge now crossed (100/500/1000 → Rising Star/Campus Hero/MANIT Legend) → persist only the missing ones with `$addToSet`. **Why:** wrapped in try/catch so it **never throws** into the caller — a points hiccup must not fail the upload/answer that earned them; `$addToSet` dedupes badges.

`createNotification(userId,type,title,desc,relatedId,relatedModel) — utils/notifications.js`

Model: Notification · **Calls:** sendPushToUser

Does: The single choke-point for notifications. **Flow:** `Notification.create({ … })` → `sendPushToUser(userId,title,desc)` to mirror it to FCM → return the doc (returns `null` on failure). **Why:** every feature calls one helper; the polymorphic `relatedId` / `relatedModel` lets the bell deep-link to a Listing/Conversation/etc.

`sendPushToUser(userId,title,body) — utils/push.js`

Libs: firebase-admin (via config) · **Model:** DeviceToken

Does: Fans an FCM push out to all of a user's devices. **Flow:** no-op if `messaging` is null → load the user's `DeviceToken`s → `messaging.sendEachForMulticast({tokens,notification,data})` → inspect each per-token result and `deleteMany` the ones FCM marks `not-registered` / `invalid-argument`. **Why: self-healing token store** — dead tokens are pruned automatically; fire-and-forget so push never breaks the triggering action.

20.4 Real-time layer (`socket/`)

`chatSocket(io) — socket/chat.socket.js (handshake)`

Libs: jsonwebtoken · **Models:** User, Conversation, Message, Notification, Friendship

Does: Authenticates every socket. **Flow:** `io.use` reads `socket.handshake.auth.token` → `jwt.verify` → `User.findById` → attach `socket.user` or reject. **Why:** the WebSocket reuses the exact same JWT as REST — one identity scheme for both transports.

`connection handler & events — socket/chat.socket.js`

Does: On connect: join personal room `user:<uid>`, register presence, then **synchronously** attach all event handlers *before* any `await`. **Events:** `join_conversation` (verify participant+tenant → `socket.join(id)`); `send_message` (re-verify → `Message.create` → update `lastMessage` → `io.to(room).emit("receive_message")` → upsert one coalesced unread notification per recipient); `mark_read` (set `readAt` on the other party's unread msgs → broadcast `messages_read`); `disconnect` (presence cleanup → emit `friend_offline` on the last socket). **After** handlers: async presence snapshot + `friend_online` announce. **Why:** Socket.IO doesn't buffer events for unattached handlers — awaiting first would silently drop a client's immediate `join_conversation`. Notifications are coalesced (one unread per conversation) so 50 messages ≠ 50 bells.

`presence registry — socket/presence.js`

Does: In-memory `Map<userId, Set<socketId>>`. `addSocket` returns "became online" only on the **first** socket; `removeSocket` returns "became offline" only on the **last**; `isOnline` / `onlineAmong` query it. **Why:** a user with 3 tabs is one "online" → online/offline pings fire once, not per tab. Per-process & ephemeral — on multi-instance/serverless it degrades to "everyone offline" (fix: Socket.IO Redis adapter).

20.5 Scheduled jobs (`jobs/`)

`classReminderJob() — jobs/classReminder.job.js`

Libs: node-cron · **Model:** TimetableEntry · **Calls:** createNotification

Does: Every 10 min, reminds about classes starting within 30 min. **Flow:** compute `istNow()` (UTC + 5.5h) → derive today / dayOfWeek / "HH:mm" → query entries in the 30-min window with `lastReminderDay≠today` → notify each → stamp `lastReminderDay=today`. **Why:** the IST offset makes it timezone-independent (hosts run UTC); `lastReminderDay` is an **idempotency key** → exactly one reminder per class per day despite a 10-min sweep.

`eventReminderJob() · studyGroupReminderJob() — jobs/*.job.js`

Libs: node-cron · **Models:** Event / StudyGroup

Does: Every 10 min, remind RSVP'd attendees (+organizer) / group members about something starting within the hour. **Flow:** query `startAt|nextSession.at` in [now, now+1h] with the "reminder not yet sent" flag false → notify recipients (study-group respects `notificationPreferences.studyGroups`) → flip `reminderSent` / `notifications.sessionReminderSent=true`. **Why:** the boolean flag is the same idempotency idea as `lastReminderDay`, scoped to the single event/session.

20 · Appendix (cont.) — Authentication & accounts

20.6 Auth (`auth/`)

`POST /api/auth/signup — auth/signup.js`

Libs: jsonwebtoken, bcryptjs (via model) · **Helpers:** resolveUniversityByEmail, generateUniqueHandle, generateToken

Does: Registers a new student. **Flow:** require email/password/displayName → reject duplicate email → `resolveUniversityByEmail` (find-or-create tenant from domain) → `generateUniqueHandle` → `User.create` (password auto-hashed by the model's `pre('save')`) → `generateToken` → 201 with `{user, token}`. **Why:** tenant is derived from the verified email domain, never chosen by the client — un-spoofable multi-tenancy.

`POST /api/auth/login — auth/login.js`

Libs: bcryptjs (via model) · **Helpers:** generateToken, generateUniqueHandle

Does: Authenticates. **Flow:** `User.findOne({email}).select("+password").populate("university")` → `user.comparePassword` (bcrypt) → return the **same generic "Invalid credentials"** for unknown-email and wrong-password → backfill a handle for legacy accounts → sign & return JWT + user. **Why:** identical error message blocks email enumeration; `+password` is needed because the field is `select:false`.

`POST /api/auth/forgot-password — auth/forgotPassword.js`

Libs: node `crypto`

Does: Starts a password reset. **Flow:** find the user → `crypto.randomBytes(32)` raw token → store only its **SHA-256 hash** + a 30-min expiry → always reply "if the email exists…" → in non-prod return the raw token in the body (no email integration is wired). **Why:** storing the hash means a DB leak can't be turned into a working reset link; the generic reply prevents enumeration.

`POST /api/auth/reset-password — auth/resetPassword.js`

Libs: node `crypto`

Does: Completes the reset. **Flow:** SHA-256 the incoming token → find a user whose stored hash matches **and** whose expiry is in the future → set the new password (re-hashed by the model hook) → clear the reset fields. **Why:** matching the hash + an unexpired window is what makes the one-time link safe.

20.7 Users (`users/`) — 16 endpoints

`GET /api/users/me — users/getMe.js`

Does: Returns the logged-in user's core profile (from the already-loaded `req.user`), lazily backfilling a `handle` for legacy accounts. **Why:** no extra DB read — `auth` already loaded the user.

`PUT /api/users/me — users/updateProfile.js`

Helpers: isValidHandle

Does: Edits displayName/phone/bio/location/handle/email. **Flow:** assign provided fields → if handle changed: validate format + uniqueness (`User.exists`) → if email changed: ensure not taken → `save`. **Why:** handle/email are uniqueness-checked against everyone except yourself (`_id:{$ne}`).

`PUT /api/users/avatar — users/uploadAvatar.js`

Middleware: uploadAvatar (disk)

Does: Uploads a profile picture. **Flow:** Multer writes to `uploads/avatars/` → set `avatarUrl = <protocol>://<host>/uploads/avatars/<filename>` → save. **Why:** the only disk-served asset (see §20.2 avatar caveat); URL is built from the request host so it works across environments.

`PUT /api/users/payment-qr — users/uploadPaymentQr.js`

Middleware: uploadPaymentQr (Cloudinary)

Does: Uploads the seller's UPI QR image to Cloudinary and stores `paymentInfo.upiQrUrl = req.file.path`. **Why:** a fallback for sellers who'd rather show a saved QR than type a UPI ID (the checkout modal prefers the ID, falls back to this image).

`PUT /api/users/payment-info — users/updatePaymentInfo.js`

Does: Saves `paymentInfo.upiId` / `upiQrUrl`. **Why:** the UPI ID is regex-validated at the schema level; this powers the client-side UPI deep link + QR.

`GET /api/users/settings · PUT /api/users/notification-preferences · PUT /api/users/privacy-settings — users/get/update*.js`

Does: Read all settings; patch notification prefs (messages/studyGroups/marketplace booleans) or privacy (profileVisibility, showOnlineStatus, allowDirectMessages). **Flow:** assign only the fields present in the body → `save`. **Why:** these booleans gate real behaviour — e.g. `showOnlineStatus` is honoured by the socket presence layer, `studyGroups` by the reminder cron.

`POST /api/users/saved-listings/:listingId — users/toggleSavedListing.js`

Does: Toggles a listing in the wishlist. **Flow:** find the id in `req.user.savedListings` → splice out or push in → save → return the array. **Why:** a single endpoint for add+remove; state lives on the user doc.

`GET /api/users/me/saved-listings — users/getSavedListings.js`

Does: Returns the wishlist's still-active, same-tenant listings, newest first, seller populated. **Why:** filters by `_id: {$in:savedListings}` + tenant so a saved-but-deleted or cross-campus item never shows.

`GET /api/users/search?q= — users/searchUsers.js`

Does: Same-university people search. **Flow:** require ≥2 chars → **escape** the regex → match handle by prefix OR displayName case-insensitively → limit 20 → annotate each with the `friendStatus` relative to me (none/outgoing/incoming/friends). **Why:** regex is escaped to prevent injection; the friendship annotation is tolerant of the model not existing (pre-Phase-2 safe).

`GET /api/users/handle-available?handle= — users/checkHandle.js`

Does: Live handle-availability check for the settings form. **Flow:** validate format → `User.exists({handle, _id: {$ne:me}})` → `{available, reason?}`. **Why:** instant feedback before save; excludes yourself so keeping your own handle reads as available.

`GET /api/users/:id/profile — users/getUserProfile.js`

Guards: auth, scope(User) · **Models:** Listing, Document, Question, Answer, Event, Ride, Friendship

Does: Public profile + contribution stats. **Flow:** `universityScope(User)` gives the profile owner → `Promise.all` of the owner's active listings + counts (docs/questions/answers/events/rides) + the friendship link → derive `friendStatus` (self/none/outgoing/incoming/friends). **Why:** all six aggregates run in parallel; `universityScope` guarantees you can only view same-campus profiles.

`GET /api/users/:id/friends — users/getFriendsOf.js`

Guards: auth, scope(User) · **Model:** Friendship

Does: Another user's friends list for the profile view. **Flow:** load accepted friendships → map each to "the other person" → if the owner's profile is `private` and the viewer is neither the owner nor a friend, hide the list but still return the **count**. **Why:** respects `profileVisibility` while keeping the count public.

`DELETE /api/users/me — users/deleteMe.js`

Model: Listing

Does: Account deletion. **Flow:** `Listing.deleteMany({seller:me})` → `req.user.deleteOne()`. **Why:** removes the user's marketplace footprint with the account (a minimal cascade; other content is left).

20.8 Universities · Dashboard · Leaderboard

`GET /api/universities — universities/getUniversities.js`

Does: Public, read-only list of tenants (name/domains), sorted by name. **Why:** the only un-authed data route — handy for a future campus picker.

`GET /api/dashboard/summary — dashboard/getDashboardSummary.js`

Models: Listing, StudyGroup

Does: Home-screen aggregates. **Flow:** `Promise.all` of active-available listing count + active study-group count, plus a campus-locations number (`CAMPUS_LOCATIONS_COUNT` env, default 85). **Why:** two scoped counts in parallel; the locations figure is config so it can be tuned without a deploy.

`GET /api/leaderboard — leaderboard/getLeaderboard.js`

Model: User

Does: Campus karma board + your rank. **Flow:** `Promise.all` of [top 20 by `points` desc, tie-break `createdAt`] + [count of users with more points than me → my rank = count+1] + [my own row]. **Why:** rank without a full sort — just count how many beat you; sorts on the indexed `points` field, scoped to the university.

20 · Appendix (cont.) — Marketplace, Offers & Study Vault

20.9 Listings (`listings/`) — 8 endpoints

`GET /api/listings – listings/getAllListings.js`

Does: The canonical paginated/filtered list. **Flow:** cap `limit` at 50 → build the query (tenant + `isActive` + default `status:available`, optional category/condition/seller/price-range/`$regex` search) → pick sort from a **whitelist map** (newest/oldest/price/title) → `Promise.all([find().populate(seller).skip().limit(), countDocuments])` → `{data, meta}`. **Why:** server-capped page size, parallel count, and a sort whitelist (never raw user input) — the three list-endpoint best practices.

`GET /api/listings/me – listings/getMyListings.js`

Does: The caller's own active listings, newest first. **Why:** mounted before `/:id` so `/me` isn't captured as an id.

`GET /api/listings/:id – listings/getListingById.js`

Guards: auth, scope(Listing)

Does: One listing (tenant-checked by `universityScope`), seller populated, 404 if inactive. **Why:** the scope middleware already loaded + tenant-verified it via `req.resource`.

`POST /api/listings – listings/createListing.js`

Helpers: createNotification, awardPoints

Does: Creates a listing. **Flow:** validate title/price/category → `Listing.create` stamping `seller` + `university` from `req.user` → self-notification → `awardPoints("listing_created")` (+5) → 201. **Why:** owner/tenant come from the token, never the body.

`POST /api/listings/:id/images – listings/uploadListingImages.js`

Guards: auth, scope(Listing) · **Middleware:** uploadListingImages.array("images",6)

Does: Appends up to 6 Cloudinary image URLs. **Flow:** scope + owner check → map `req.files→file.path` → append to `images` → save. **Why:** images stream to Cloudinary (resized) and only URLs are stored.

`PUT /api/listings/:id · PATCH /api/listings/:id/status · DELETE /api/listings/:id – listings/update*/delete*.js`

Guards: auth, scope(Listing)

Does: Owner-only edit (`Object.assign`), status change (validated against `available|sold|reserved`), and **soft delete** (`isActive=false`). **Why:** all three repeat the same owner check and 404 with an identical message (don't reveal a resource exists to non-owners); delete is soft so references stay intact.

20.10 Offers (`offers/`) — the negotiation state machine

`POST /api/offers — offers/createOffer.js`

Helpers: POPULATE, formatINR, createNotification

Does: Buyer makes an offer. **Flow:** validate amount ≥1 → load listing (must be active & not sold, same tenant) → **reject self-offer** → reject a **duplicate active offer** (`findOne status∈{pending,countered}`) → `Offer.create` with `seller` denormalized from the listing → notify seller with ₹-formatted amount → 201. **Why:** guard rails enforce the real-world rules; denormalizing `seller` makes "offers I received" one indexed query.

`GET /api/offers?role=buyer|seller — offers/getOffers.js`

Does: Lists offers from either side. **Flow:** tenant + (`seller=me` or `buyer=me`) + optional listing filter → populate listing/buyer/seller → newest-updated first. **Why:** one endpoint serves both the "Offers I made" and "Offers on my items" tabs.

`PATCH /api/offers/:id — offers/updateOffer.js`

Guards: auth, scope(Offer) · **Helpers:** POPULATE, formatINR, createNotification

Does: Drives every transition. **Flow:** identify the viewer as buyer/seller → restrict actions (seller: accept/decline/counter; buyer: withdraw/accept-counter/decline-counter) → require the offer be still active (`pending|countered`) → `switch` on action: set status, on **accept/accept-counter** reserve the listing (`status:reserved`), copy `counterAmount→amount` on accept-counter → notify the counterparty. **Why:** a guarded state machine — illegal transitions are rejected, and an accept reserves the item so two buyers can't both win.

`offers/populate.js · offers/formatINR.js (shared)`

Does: `POPULATE` = the listing/buyer/seller projection reused by all three offer endpoints; `formatINR(n)` = `₹` + `toLocaleString("en-IN")` (Indian 1,20,000 grouping). **Why:** DRY — one projection & one formatter, imported everywhere.

20.11 Study Vault / Documents (`documents/`) — 7 endpoints

`GET /api/documents — documents/getDocuments.js`

Does: Filtered, paginated notes feed. **Flow:** cap limit 50 → query tenant + filters (type/branch/semester exact, subject + search via `$regex`) → sort whitelist (newest/oldest/**downloads**/**top**=upvoteCount/title) → `Promise.all([find().populate(uploader, comments.author), count])` → **`serializeDocument`** each: add boolean `myUpvote` , delete the raw `upvotes[]` . **Why:** the "Top rated" sort hits the indexed `upvoteCount` ; stripping `upvotes[]` protects voter privacy and shrinks the payload.

`GET /api/documents/me — documents/getMyDocuments.js`

Does: The caller's own uploads, newest first. **Why:** mounted before `/:id` routes.

`POST /api/documents — documents/createDocument.js`

Middleware: uploadDocument.single("file") · **Helpers:** awardPoints

Does: Uploads a study file. **Flow:** Multer streams the file to Cloudinary → validate title/type + presence of `req.file` → derive extension from the original name → `Document.create` storing `fileUrl(=secure_url)/fileName/fileFormat/fileSize` + uploader/tenant → `awardPoints("document_upload")` (+15, "Note Sharer"). **Why:** the DB holds only the URL + metadata — the binary lives on Cloudinary's CDN (this is the "how are PDFs stored" answer, Part 9).

`POST /api/documents/:id/download — documents/incrementDownload.js`

Guards: auth, scope(Document)

Does: `$inc downloadCount` and returns the `fileUrl`. **Why:** a cheap analytics/sort signal; the actual bytes are fetched from Cloudinary, not the API.

`POST /api/documents/:id/upvote — documents/toggleUpvote.js`

Guards: auth, scope(Document)

Does: Toggles the viewer's upvote. **Flow:** add/remove from `upvotes[]` → keep `upvoteCount=upvotes.length` in sync → return `{upvoteCount, myUpvote}`. **Why:** maintaining the denormalized counter is what makes "Top rated" sort cheap.

`POST /api/documents/:id/comments — documents/addComment.js`

Guards: auth, scope(Document) · **Helpers:** createNotification

Does: Adds a comment (≤500 chars) and notifies the uploader (unless self). **Flow:** validate → push into embedded `comments[]` → save → populate the new comment's author → return it. **Why:** comments are embedded (always read with the doc, bounded).

`DELETE /api/documents/:id — documents/deleteDocument.js`

Guards: auth, scope(Document)

Does: Uploader-only soft delete (`isActive=false`). **Why:** soft delete keeps comment/upvote references valid and lets it vanish from feeds.

20 · Appendix (cont.) — Study Groups, Chat & Confessions

20.12 Study Groups (`studyGroups/`) — 10 endpoints

`GET /api/study-groups – studyGroups/getAllStudyGroups.js`

Does: Paginated group list. **Flow:** cap limit 50 → tenant + `isActive` + optional subject + `$text` search + `myGroups` (members=me) → populate creator → newest first → `{groups, meta}`. **Why:** uses a real **MongoDB `$text` index** here (name/description/subject/tags), unlike the regex search elsewhere.

`GET /api/study-groups/upcoming – studyGroups/getUpcomingSessions.js`

Does: Groups whose `nextSession.at ≥ now`, soonest first (cap 20). **Why:** registered *before* `/:id` so "upcoming" isn't read as an id — the classic static-before-param ordering.

`GET /api/study-groups/:id – studyGroups/getStudyGroupById.js`

Guards: auth, scope(StudyGroup)

Does: One group with creator + members populated. **Why:** scope verifies tenant first, then a fresh populated fetch builds the detail view.

`POST /api/study-groups – studyGroups/createStudyGroup.js`

Helpers: createNotification

Does: Creates a group with the creator as first member. **Flow:** validate name/subject → normalise `nextSession` (accepts either a nested object or a flat `nextSessionAt`) → `create` with `creator` + `members:[me]` → self-notification. **Why:** the model's `pre('save')` also guarantees the creator is a unique member.

`POST /api/study-groups/:id/join · /leave – studyGroups/joinStudyGroup.js · leaveStudyGroup.js`

Guards: auth, scope(StudyGroup) · **Helpers:** createNotification (join)

Does: Join enforces "not already a member" + **member cap** (`members.length ≥ maxMembers`) then notifies the creator; leave blocks the **creator** from leaving and removes a member. **Why:** capacity and the creator-can't-leave rule keep the group consistent.

`PUT /api/study-groups/:id · /:id/next-session – updateStudyGroup.js · updateNextSession.js`

Guards: auth, scope(StudyGroup)

Does: Creator-only edits to group fields / the next session. **Key detail:** whenever the session changes, reset `notifications.sessionReminderSent=false`. **Why:** resetting the flag re-arms the reminder cron for the new time (idempotency reset).

`POST /api/study-groups/:id/cover · DELETE /api/study-groups/:id – uploadGroupCover.js · deleteStudyGroup.js`

Guards: auth, scope(StudyGroup) · **Middleware:** uploadStudyGroupCover

Does: Creator-only cover-image upload (Cloudinary 1400×800) and soft delete (`isActive=false`). **Why:** same owner-check + soft-delete pattern as the rest of the app.

20.13 Conversations & Messages (`conversations/` , `messages/`)

`POST /api/conversations — conversations/createConversation.js`

Models: Conversation, Listing, User, Friendship · **Helpers:** createNotification

Does: Find-or-create a chat. **Flow:** a `contextLoaders` map handles **listing/lostfound/ride** (load the context doc, tenant-check, derive `listingTitle` server-side) → sort the participant pair → `findOne` existing or `create` → notify the other party. A separate `startFriendConversation` branch handles **friend DMs** (no listing; requires an *accepted* Friendship). **Why:** one polymorphic Conversation model serves four chat sources; the title is derived (never trusted from the body); the find-or-create + unique index makes "message seller" idempotent.

`GET /api/conversations — conversations/getUserConversations.js`

Models: Conversation, Message

Does: The inbox. **Flow:** find conversations where I'm a participant (tenant-scoped) → populate participants → sort by `lastMessageAt` → for each, `countDocuments` unread (others' messages with `readAt:null`) → attach `unreadCount`. **Why:** the denormalized `lastMessage/lastMessageAt` render the list without touching Messages; only the per-row unread count needs a query.

`GET /api/messages/:conversationId — messages/getMessages.js`

Guards: auth, scope(Conversation,"conversationId")

Does: Paginated history. **Flow:** scope + participant check → cap limit 100 → fetch newest-first with skip/limit, then `.reverse()` to chronological → `{messages, meta}`. **Why:** uses the `{conversation, createdAt}` index; fetch-desc-then-reverse gives the latest page cheaply in reading order.

`POST /api/messages/:conversationId — messages/sendMessage.js`

Guards: auth, scope(Conversation) · **Models:** Message, Notification

Does: The REST twin of the socket's `send_message`. **Flow:** validate content → participant check → `Message.create` → update `lastMessage` → if an `io` is attached, `io.to(room).emit("receive_message")` → upsert one coalesced unread notification per recipient. **Why:** chat still works without a live socket (serverless/polling); identical logic to the socket handler so behaviour matches either path.

`PUT /api/messages/:conversationId/read — messages/markConversationRead.js`

Guards: auth, scope(Conversation)

Does: Marks the other party's messages read (`readAt=now`). **Why:** a REST path for read receipts so they work even when the socket's `mark_read` isn't connected.

20.14 Confessions (`confessions/`) — 6 endpoints + serializer

`serializeConfession(confession, viewerId) — confessions/serializeConfession.js`

Libs: node `crypto` (SHA-1)

Does: Maps a confession to its **anonymised** public shape. **Flow:** tally reaction counts + the viewer's `myReaction` → for each comment, the handle is `"OP"` for the original poster else `anonHandle = "Anon-"+sha1(`${confessionId}:${userId}:salt`).slice(0,4)` → expose `isMine` / `reportedByMe` but **never the author id**. **Why:** the handle is stable in-thread, uncorrelated across threads, and irreversible — provable anonymity without storing anything extra.

`GET /api/confessions — confessions/getConfessions.js`

Does: The feed. **Flow:** cap limit 30 → query tenant + `isActive` + `isHidden:false` → sort newest or **top** (reactionsCount) → map through `serializeConfession`. **Why:** the compound index `{university,isActive,isHidden,createdAt}` matches this exact query+sort; hidden (auto-moderated) posts never appear.

`POST /api/confessions — confessions/createConfession.js`

Helpers: awardPoints, serializeConfession

Does: Posts an anonymous confession (≤1000 chars). **Flow:** validate → `create` storing the real `author` (privately) → `awardPoints("confession_posted")` (+2) → return the **serialized** (author-stripped) doc. **Why:** author is stored for moderation but the response is already anonymised.

`POST /api/confessions/:id/react — confessions/react.js`

Guards: auth, scope(Confession)

Does: Toggle/switch a reaction (heart/laugh/wow/sad). **Flow:** validate type against `REACTION_TYPES` → if same type exists remove it, if different replace it, else add → resync `reactionsCount` → return serialized. **Why:** one reaction per user, switchable; counter kept in sync for the "top" sort.

`POST /api/confessions/:id/comments — confessions/addComment.js`

Guards: auth, scope(Confession) · **Helpers:** createNotification

Does: Anonymous threaded reply (≤500). **Flow:** validate → push into embedded `comments[]` → notify the OP ("someone replied" — no identity) unless self → return serialized. **Why:** the notification deliberately leaks nothing about who replied.

`POST /api/confessions/:id/report — confessions/report.js`

Guards: auth, scope(Confession)

Does: Community moderation. **Flow:** block duplicate reports by the same user → push `{user, reason}` → if `reports.length ≥ 5` set `isHidden=true`. **Why:** the threshold (5) **auto-hides** abusive posts without waiting for an admin; admins can still act via the Reports queue.

`DELETE /api/confessions/:id — confessions/deleteConfession.js`

Guards: auth, scope(Confession)

Does: Author-only soft delete (`isActive=false`). **Why:** author identity is checked server-side against the privately stored `author`.

20 · Appendix (cont.) — Q&A Forum, Rides, Events, Lost & Found

20.15 Course Q&A Forum (`forum/`) — 9 endpoints + helpers

`forum/helpers.js (shared)`

Does: `AUTHOR_FIELDS` = the author projection; `withMyUpvote(doc, viewerId)` = add boolean `myUpvote` and **delete the raw** `upvotes[]` from the payload. **Why:** the client only needs the count + "did I vote" — shipping every voter id leaks identities and grows unbounded.

`GET /api/forum/questions — forum/getQuestions.js`

Does: Paginated question list. **Flow:** cap limit 30 → query tenant + filters (branch/semester exact, subject + search via `$regex`, `unanswered` →answersCount:0) → sort whitelist (newest/top/unanswered) → populate author → map `withMyUpvote`. **Why:** same list pattern; "top" sorts on indexed `upvoteCount`.

`GET /api/forum/questions/:id — forum/getQuestion.js`

Guards: auth, scope(Question)

Does: A question + its answers. **Flow:** populate author → load active answers sorted by `upvoteCount` → mark each `isAccepted` → re-sort so the **accepted answer floats to the top** → return both via `withMyUpvote`. **Why:** StackOverflow ordering — accepted first, then by votes.

`POST /api/forum/questions — forum/createQuestion.js`

Helpers: awardPoints, withMyUpvote

Does: Posts a question. **Flow:** require title → `create` (author/tenant from token) → populate author → `awardPoints("question_posted")` (+5). **Why:** tagged by branch/subject/semester for filtering.

`POST /api/forum/questions/:id/answers — forum/addAnswer.js`

Guards: auth, scope(Question) · **Helpers:** createNotification, awardPoints

Does: Adds an answer. **Flow:** validate body → `Answer.create` → `$inc question.answersCount` → notify the asker (unless self) → `awardPoints("answer_posted")` (+10). **Why:** `answersCount` is denormalized on the question so lists don't count answers each render.

`POST /api/forum/questions/:id/upvote · /answers/:id/upvote — upvoteQuestion.js · upvoteAnswer.js`

Guards: auth, scope(Question|Answer)

Does: Toggle the viewer's upvote and resync `upvoteCount` → return `{upvoteCount, myUpvote}`. **Why:** identical toggle-and-sync pattern as documents; powers the "top" sorts.

`POST /api/forum/answers/:id/accept — forum/acceptAnswer.js`

Guards: auth, scope(Answer) · **Helpers:** createNotification, awardPoints

Does: Question author marks/un-marks the solution (toggle). **Flow:** load the answer's question → assert caller is the question author → toggle `acceptedAnswer` → on accept: `awardPoints(answer.author,"answer_accepted")` (+25, "Problem Solver") + notify. **Why:** only the asker can accept; toggling lets them change their mind (points only granted on first accept).

`DELETE /api/forum/questions/:id · /answers/:id — deleteQuestion.js · deleteAnswer.js`

Guards: auth, scope(Question|Answer)

Does: Author-only soft delete. Deleting an answer also `$dec answersCount` and clears `acceptedAnswer` if it was the accepted one. **Why:** keeps the question's denormalized counters and accepted-answer pointer correct.

20.16 Ride Share (`rides/`) — 5 endpoints

`rides/populate.js (shared)`

Does: The poster + passengers projection reused by all ride endpoints.

`GET /api/rides — rides/getRides.js`

Does: Lists rides. **Flow:** filter `upcoming` (departure ≥ now) or `mine` (poster OR passenger) → optional from/to search → sort by departure → paginate. **Why:** "mine" uses `$or` across poster+passengers so your booked rides show too.

`POST /api/rides — rides/createRide.js`

Helpers: awardPoints

Does: Posts a trip. **Flow:** validate from/to/departure/seats → reject past departure → seats 1–8 → `create` → `awardPoints("ride_posted")` (+5, "Road Tripper"). **Why:** future-date + seat-range validation up front.

`POST /api/rides/:id/join · /leave — joinRide.js · leaveRide.js`

Guards: auth, scope(Ride) · **Helpers:** createNotification

Does: Seat accounting. Join: reject if departed / your own ride / already joined / **no seats left** → push passenger → notify poster. Leave: remove from passengers (error if you weren't in) → notify poster. **Why:** capacity is enforced against `seatsTotal`; both sides notify the poster.

`DELETE /api/rides/:id — rides/deleteRide.js`

Guards: auth, scope(Ride) · **Helpers:** createNotification

Does: Poster-only soft delete that **notifies every booked passenger** "ride cancelled". **Why:** people who reserved a seat must hear about a cancellation (`Promise.all` over passengers).

20.17 Events & Clubs (`events/`) — 4 endpoints + helpers

`events/populate.js · events/serializeEvent.js (shared)`

Does: `POPULATE` = organizer+attendees projection; `serializeEvent` adds `attendeeCount` + `myRsvp` and trims `attendees` to a 5-avatar strip. **Why:** the card needs a count and "am I going", not the whole attendee list.

`GET /api/events — events/getEvents.js`

Does: Lists events. **Flow:** filter `upcoming` / `past` / `mine` (organizer OR attendee) → optional category + search → sort by start → paginate → `serializeEvent` each. **Why:** one endpoint, three time-views.

`POST /api/events — events/createEvent.js`

Helpers: awardPoints, serializeEvent

Does: Creates an event. **Flow:** validate title/category/start → reject past start → if `endAt` given, must be after start → `create` → `awardPoints("event_created")` (+10, "Event Host"). **Why:** chronological sanity checks before persisting.

`POST /api/events/:id/rsvp · DELETE /api/events/:id — toggleRsvp.js · deleteEvent.js`

Guards: auth, scope(Event)

Does: Toggle attendance (blocked once the event has started) and organizer-only soft delete. **Why:** RSVP is a single toggle; the event-reminder cron later pings everyone in `attendees`.

20.18 Lost & Found (`lostFound/`) — 4 endpoints

`GET /api/lost-found — lostFound/getItems.js`

Does: Board of lost/found items. **Flow:** filter by kind (lost/found), category, status (open/returned) + search (title/description/location) → paginate → populate reporter. **Why:** the `kind` + `status` filters drive the two board tabs.

`POST /api/lost-found — lostFound/createItem.js`

Middleware: uploadLostFoundImages.array("images",4) · **Helpers:** awardPoints

Does: Posts an item with up to 4 Cloudinary photos. **Flow:** validate title/kind/category → map files to URLs → `create` → `awardPoints("lostfound_posted")` (+5). **Why:** posters are reachable via chat (Conversation `contextType:"lostfound"`).

`PATCH /api/lost-found/:id/status · DELETE /api/lost-found/:id — updateStatus.js · deleteItem.js`

Guards: auth, scope(LostFoundItem)

Does: Reporter-only status flip (`open` ↔ `returned`) and soft delete. **Why:** only the person who posted can mark their item returned or remove it.

20 · Appendix (cont.) — Academics, Notifications, Friends, Moderation

20.19 Timetable (`timetable/`) — 4 endpoints + validator

`validateEntry({subject,dayOfWeek,startTime,endTime}) — timetable/validateEntry.js`

Does: Shared validation: subject required, day 0–6, times match `HH:mm`, `endTime > startTime`. **Why:** one validator imported by create + update so the rules can't drift.

`GET /api/timetable · POST /api/timetable — getEntries.js · createEntry.js`

Does: List the user's classes (sorted day→time) and add one. **Flow (create):** `validateEntry` → `create` with user/tenant. **Why:** the class-reminder cron reads these by `dayOfWeek` + `startTime`.

`PUT /api/timetable/:id · DELETE /api/timetable/:id — updateEntry.js · deleteEntry.js`

Guards: auth, scope(TimetableEntry)

Does: Owner-only edit (merge + re-validate) and hard delete. **Key detail:** update sets `lastReminderDay=null`. **Why:** resetting the marker re-arms a reminder for the (possibly new) slot today.

20.20 Attendance (`attendance/`) — 4 endpoints + guard

`ownedByUser(subject, userId) — attendance/ownedByUser.js`

Does: One-liner ownership check reused by update/delete. **Why:** tiny shared guard so both endpoints reject other users' subjects identically.

`GET /api/attendance · POST /api/attendance — getSubjects.js · createSubject.js`

Does: List the user's subjects (with attended/held/target) and add one. **Flow (create):** validate name, clamp attended/held ≥0, reject attended>held, target 1–100. **Why:** the invariant "attended ≤ held" is enforced on the way in.

`PATCH /api/attendance/:id — attendance/updateSubject.js`

Guards: auth, scope(AttendanceSubject)

Does: Two modes in one route. **Action mode:** `present` (+attended,+held), `absent` (+held), `undo-present`, `undo-absent` (with "nothing to undo" guards). **Edit mode:** set name/attended/held/target with re-validation. **Why:** the daily one-tap buttons and manual corrections share one endpoint; undo guards stop counters going negative.

`DELETE /api/attendance/:id — attendance/deleteSubject.js`

Guards: auth, scope(AttendanceSubject)

Does: Owner-only hard delete (via `ownedByUser`). **Why:** attendance is private per-user data — no soft-delete needed.

20.21 Academic Records / CGPA (`academicRecords/`) — 3 endpoints

`GET /api/academic-records – academicRecords/getRecords.js`

Does: All of the user's semesters (subjects + grades), ordered by semester. **Why:** the client computes SGPA/CGPA from this raw data (server stays the source of truth for inputs, not derived numbers).

`PUT /api/academic-records/:semester – academicRecords/upsertSemester.js`

Does: Create-or-replace a semester. **Flow:** validate semester 1–12 → `validateSubjects` (name, credits 0.5–30, grade ∈ A+…F) → `findOneAndUpdate({user,semester}, {$set:subjects, $setOnInsert:user/university/semester}, {upsert, runValidators})`. **Why:** one idempotent upsert handles both first save and edits; the unique `{user,semester}` index prevents duplicate semesters.

`DELETE /api/academic-records/:semester – academicRecords/deleteSemester.js`

Does: Removes a semester record (validates 1–12, 404 if absent). **Why:** keyed by semester, not `_id` — the UI thinks in semesters.

20.22 Notifications (`notifications/`) — 4 endpoints

`GET /api/notifications – notifications/getNotifications.js`

Does: The bell list. **Flow:** filter by type/read → `Promise.all([find().sort(-createdAt).paginate, total, unreadCount])` → `{notifications, meta(+unreadCount)}`. **Why:** the `unreadCount` in meta is what the bell badge polls (`useUnreadCount` fetches limit:1 just for this number).

`PUT /:id/read · PUT /read-all · DELETE /:id – markAsRead.js · markAllAsRead.js · deleteNotification.js`

Does: Mark one read, mark all read (`updateMany`), delete one — all scoped to `user:req.user._id`. **Why:** every query is user-scoped so you can only touch your own notifications.

20.23 Push tokens (`push/`) — 2 endpoints

`POST /api/push/register – push/registerToken.js`

Model: DeviceToken

Does: Registers a device for FCM. **Flow:** validate token → `findOneAndUpdate({token}, {token,platform,user,university}, {upsert})`. **Why:** upsert-by-token **re-assigns a shared device** to the current user (so a borrowed phone doesn't get someone else's pushes). Web and Android hit the same endpoint with a `platform` tag.

`POST /api/push/unregister – push/unregisterToken.js`

Does: Deletes the token for the current user on logout. **Why:** scoped to `{token, user}` so you can only remove your own registration.

20.24 Friends (`friends/`) — 6 endpoints + helpers

`friends/helpers.js (shared)`

Does: `PUBLIC_FIELDS` = the safe friend projection; `sortedPair(a,b)` = the canonical sorted id pair. **Why:** sorting the pair is what makes the friendship unique-index and lookups order-independent (`[A,B]===[B,A]`).

`GET /api/friends — friends/getFriends.js`

Calls: presence.isOnline

Does: Accepted friends + live online status. **Flow:** load accepted links → map to "the other person" → `isOnline = friend's showOnlineStatus && presence.isOnline(id)` . **Why:** presence is read from the in-memory registry and gated by the friend's own privacy choice.

`GET /api/friends/requests — friends/getRequests.js`

Does: Splits pending links into `{incoming, outgoing}` by comparing `requester` to me. **Why:** one query, two buckets for the UI.

`POST /api/friends/requests/:userId — friends/sendRequest.js`

Helpers: createNotification, sortedPair

Does: Sends a request. **Flow:** validate id, block self, target must exist + same tenant → reject if a link already exists → `create` pending with the sorted pair → notify recipient. **Catch:** a unique-index race (`err.code 11000`) is mapped to "already exists". **Why:** the `pairKey` unique index plus the 11000 handler make concurrent double-requests safe.

`PATCH /api/friends/requests/:userId/accept — friends/acceptRequest.js`

Helpers: createNotification, sortedPair

Does: Accepts an incoming request. **Flow:** find the pending link → assert I'm the **recipient** → set `status:accepted` → notify the requester. **Why:** only the recipient can accept (not the sender).

`DELETE /api/friends/requests/:userId · DELETE /api/friends/:userId — removeRequest.js · unfriend.js`

Does: Decline/cancel a **pending** link, or remove an **accepted** one. **Why:** two routes keyed on status — one cancels a request, the other ends a friendship.

20.25 Reports & moderation (`reports/`) — 3 endpoints + registry

`reports/targets.js (shared registry)`

Does: Maps each reportable type (listing/document/confession/question/answer/lostfound/ride/event) to `{model(), snapshot(doc), remove(doc)}` . **Why:** a strategy table — one polymorphic Report model handles 8 content types; `snapshot` copies a title/excerpt so the queue stays reviewable even after removal; `remove` knows the right "hide" per type (`isActive=false` vs confession's `isHidden=true`).

`POST /api/reports — reports/createReport.js`

Does: Files a report. **Flow:** look the type up in `TARGETS` → load the doc, tenant-check → block duplicate report by the same user (unique index too) → `Report.create` with a content `snapshot` . **Why:** the snapshot survives deletion of the original.

`GET /api/reports — reports/getReports.js`

Guards: auth, isAdmin

Does: Admin moderation queue. **Flow:** filter by status (default open) → paginate (cap 50) → populate reporter/handledBy → return data + meta(+openCount). **Why:** `isAdmin`-gated and tenant-scoped — admins only see their own campus's queue.

`PATCH /api/reports/:id — reports/handleReport.js`

Guards: auth, isAdmin, scope(Report)

Does: Resolves a report. **Flow:** reject if already handled → `dismiss` (status:dismissed) or `remove` (load the target via `TARGETS`, call its `remove()`, save, status:resolved, then **resolve every other open report on the same target**) → stamp handledBy/handledAt. **Why:** a "remove" auto-resolves duplicate reports so the queue doesn't fill with the same content.

20 · Appendix (cont.) — the frontend functions

The React app is the *same artifact* that ships to the browser and (via Capacitor) to Android. These are the functions that make it run: the HTTP layer, global state, the realtime client, push, and the pure-math helpers behind the academic tools.

20.26 The HTTP layer

`api (axios instance) — src/api/axios.js`

Libs: axios

Does: The one shared HTTP client. **Flow:** `baseURL` resolves by priority — explicit `VITE_API_URL` → else dev uses `/api` (Vite proxy) → else the hosted Render backend. A single **request interceptor** reads the JWT from `localStorage` and sets `Authorization: Bearer …` on every call. **Why:** no endpoint ever sets the auth header by hand; the base URL adapts to dev/prod/native with zero code change.

`feature API modules — src/api/listings.js, offers.js, … (22 files)`

Does: Thin per-feature wrappers of named functions (`fetchListings`, `createListing`, `updateListingStatus`, `uploadListingImages` …), each calling `api` and returning `res.data.data` / `.meta`. File uploads build a `FormData` and set `multipart/form-data`. **Why:** screens import `listingsApi.fetchListings()` instead of sprinkling URLs/response-unwrapping everywhere — one place per feature to change if the API shifts.

20.27 Global state (React Context)

`AuthProvider / login / logout — src/context/AuthContext.jsx`

Does: Holds the current `user` in state; delegates persistence to `authStorage`. **Flow:** lazy initial state parses the saved user from `localStorage` (guarding `"undefined"` / `"null"` /corrupt JSON); `login` / `logout` set state and call `persistAuth` / `clearAuth`, and **normalise** `id` → `_id` (the API returns `user.id`, the app expects Mongo's `_id`). **Why:** defensive parsing avoids a white-screen from a bad storage value; the id normalisation prevents subtle "undefined id" bugs.

`persistAuth / clearAuth / bootstrapAuthStorage — src/utils/authStorage.js`

Libs: @capacitor/preferences

Does: The single writer for the persisted session, kept in two tiers — durable **Capacitor Preferences** (native `SharedPreferences` on Android) plus a synchronous `localStorage` mirror for the axios interceptor and socket handshake. **Flow:** `persistAuth` writes the `localStorage` mirror first (synchronous, so the token is instantly usable) then the durable copy; `clearAuth` removes both; `bootstrapAuthStorage` runs once before render (from `main.jsx`), hydrating `localStorage` from Preferences, and migrating a legacy `localStorage`-only session into Preferences if needed. **Why:** Android's WebView wipes `localStorage` when the OS kills the app, which logged users out on restart; the durable tier survives that while the mirror keeps synchronous readers working. On the web Preferences is itself `localStorage`-backed, so the second tier is a no-op there.

`ThemeProvider / setTheme / toggle — src/context/ThemeContext.jsx`

Libs: @capacitor/status-bar

Does: Light/dark theming. **Flow:** initial = saved choice → else OS `prefers-color-scheme`; an effect toggles `.dark` on `<html>`, sets `colorScheme`, persists, and **on native recolors the Android status bar** to match. **Why:** one source of truth for theme; the status-bar sync is the kind of native touch that makes the WebView feel like a real app.

`ProtectedRoute — src/components/auth/ProtectedRoute.jsx`

Libs: react-router-dom

Does: Declarative auth gate: if no `user` in context, `<Navigate to="/auth" replace/>`, else render the nested `<Outlet/>`. **Why:** wraps the whole authenticated layout in `App.jsx` so every private route is guarded in one place.

20.28 Routing & native shell

`App() — src/App.jsx`

Libs: react-router-dom, @capacitor/app, @capacitor/core

Does: The route table + native back button. **Flow:** every screen is `lazy()` + a `<Suspense>` fallback → public routes (`/`, `/auth`) outside the guard, the rest nested under `ProtectedRoute → AppLayout` → a `*` 404. A `useEffect` (native only) listens for Android `backButton`: history-back if possible, else `exitApp()`. **Why:** lazy routes = a small initial bundle (login screen doesn't pull in chat/maps); the back-button handler is the one place the app branches on `Capacitor.isNativePlatform()` for navigation.

`main.jsx — provider tree & fonts`

Does: Mounts `ThemeProvider → BrowserRouter → AuthProvider → App` and imports the self-hosted `@fontsource` fonts (Inter/Sora/JetBrains Mono). **Why:** bundling the fonts means the app renders correctly **offline** inside the Android WebView (no Google Fonts CDN call).

20.29 Realtime & push (client)

`SocketService (singleton) — src/utils/socket.js`

Libs: socket.io-client

Does: One app-wide socket. **Key methods:** `connect()` (idempotent; passes the JWT in `auth.token`; on every `'connect'` **re-joins all open conversation rooms** from a tracked `Set`), `joinConversation/leaveConversation` (maintain that set), `sendMessage/markRead` (emit), and `on/off` wrappers for `receive_message`, `messages_read`, and the three presence events. **Why:** each reconnect is a fresh server socket that has joined nothing — re-joining on `'connect'` is what stops messages silently dying after a network blip.

`initPush() — src/lib/push.js`

Libs: @capacitor/core, dynamic `firebase/*` or `@capacitor/push-notifications`

Does: Registers the device for FCM. **Flow:** run-once guard → branch on `Capacitor.isNativePlatform()`: **native** → request permission, listen for the `registration` token, `register()`; **web** → (only if Firebase config + VAPID present) request Notification permission, **dynamically import** `firebase/app` + `messaging`, register the service worker, `getToken({vapidKey})`. Both call `registerPushToken(token, platform)`. **Why:** the dynamic import keeps Firebase out of the initial bundle; the whole thing is wrapped so a denied/unconfigured push never breaks the app.

`useUnreadCount() — src/hooks/useUnreadCount.js`

Libs: react-router-dom (useLocation)

Does: The bell badge number. **Flow:** on every route change (`location.pathname` dep), fetch notifications with `{read:false, limit:1}` and read `meta.unreadCount`; an `active` flag guards against setting state after unmount. **Why:** a pragmatic "poll on navigation" — cheap and good enough for a badge, no extra socket channel needed.

20.30 Pure helpers & notable components

`cn(...inputs) — src/lib/cn.js`

Libs: clsx + tailwind-merge

Does: `twMerge(clsx(inputs))` — build a conditional class string, then resolve Tailwind conflicts so the last utility wins. **Why:** lets a base component set `px-2` and a caller override with `px-4` without both surviving.

`attendance math — src/components/attendance/attendanceMath.js`

Does: Pure functions behind "can I skip?". `skippableClasses = max(0, floor(attended·100/target − held))`; `classesToRecover = ceil((target·held − 100·attended)/(100 − target))` (with guards: `held≤0` →0, `target≥100` →Infinity). **Why:** closed-form solutions (no loops) of `attended/(held+x) ≥ target` — exactly the kind of small algebra an interviewer can ask you to derive.

`CGPA math — src/components/cgpa/constants.js`

Does: Grade→point map (A+=10…F=0); `computeSgpa` = Σ(credits×points)/Σcredits for one semester; `computeCgpa` = the same over every subject across semesters; `totalCredits`. **Why:** a credit-weighted average computed client-side for instant feedback while typing grades; the server only persists the raw subjects.

`PayUpiModal — src/components/marketplace/PayUpiModal.jsx`

Libs: qrcode

Does: UPI checkout, entirely client-side. **Flow:** load the seller's `paymentInfo` → `buildUpiLink` = `upi://pay?pa=&pn=&cu=INR&am=&tn=` (URLSearchParams) → `QRCode.toDataURL(link)` rendered as an `<img>` → a "pay" button that's an `<a href="upi://…">`; falls back to the seller's saved QR image if they have no UPI ID. **Why:** the QR is a pure function of (UPI ID + amount), so there's no server round-trip and no stored image; the standard `upi://` link opens any UPI app — "payments go directly to the seller."

`CampusMaps — src/screens/CampusMaps.jsx`

Does: Campus directory + map. **Flow:** a hand-curated list of ~40 MANIT locations (all 12 hostels with Bhawan names, departments, canteens, sports grounds, landmarks), each with a Google Maps query (place name or exact lat/long); search + category filter via `useMemo`; renders a Google **My Maps embed** (`maps/d/embed?mid=…`) and swaps to a per-place Google Maps query on selection. **Why:** a Google embed needs no API key, no tile hosting/billing, and brings live POI/satellite/Directions for free — Leaflet (in deps) is kept for a future interactive iteration.

20.31 Build & dev config

`vite.config.js — dev proxy & vendor chunks`

Does: Two jobs. **Dev:** proxy `/api` → `VITE_PROXY_TARGET` with `changeOrigin`, so the browser only ever makes same-origin calls (no CORS locally). **Build:** a `manualChunks` function splits `node_modules` into `vendor-react / -router / -motion / -socket / -axios / -icons / -misc`. **Why:** vendors change rarely → browsers cache them across deploys, so shipping a feature only invalidates the app chunk.

`capacitor.config.json — the native shell`

Does: Configures the Android app: `appId in.ac.manit.hub`, `webDir:"dist"`, navy splash, `CapacitorHttp.enabled:true` (native HTTP → bypasses browser CORS), status-bar style, `androidScheme:"https"`. **Why:** this is the file that turns the web `dist/` into a native app and is the reason the README can say "no extra CORS configuration" on Android.

> **That's every function.** Backend: 5 config/bootstrap units, 11 middleware, 7 utils, the socket layer, 3 cron jobs, and ~150 endpoints across 23 feature folders. Frontend: the axios layer, 2 contexts, the route table + native shell, the socket singleton, push, and the pure academic-math helpers. Pair this appendix with Parts 1–19: the early parts give you the *story* to tell, this part gives you the *line-level detail* to back any follow-up question.

21 · Notable fixes — before → after

The bugs that actually shipped, and how each was resolved. Every fix is framed as the question a reviewer (or user) would ask, with the **before** — what was wrong — and the **after** — what fixed it.

21.1 Why did you get logged out every time you closed the app or the site?

Q. Log in, close the app (or the browser), reopen — and you're back on the Sign-in screen. Why did it happen, and what fixed it?

There were **two independent causes**, one per platform — so the fix has two parts.

Before — web: the start route `/` renders the marketing `LandingPage`, which never read the auth context. Reopening the site always showed "Sign in / Get started" even though the JWT was still in `localStorage`. The browser *kept* the session — the app just never looked at it, so it only *appeared* logged out.

```
// ProtectedRoute guarded /dashboard, but "/" was a public marketing page
// with no notion of a logged-in user → it always rendered the Sign-in CTA.
```

Before — Android: the token + user were stored *only* in the WebView's `localStorage`. Android flushes WebView local storage lazily and wipes it when it kills the app process (swipe-close, low memory), so on the native app the session genuinely disappeared on restart.

After: two changes, both shipped.

- **Durable native storage.** A new `frontend/src/utils/authStorage.js` write-throughs the session to **Capacitor Preferences** (native `SharedPreferences`) in addition to `localStorage`. On launch, `bootstrapAuthStorage()` — awaited in `main.jsx` *before* React renders — copies the durable values back into `localStorage` so the synchronous axios interceptor and socket auth keep reading the token without going async.
- **Session-aware entry.** `LandingPage` and `Auth` now send an already-authenticated user straight into the app.

```
// LandingPage.jsx / Auth.jsx
const { user } = useAuthContext();
if (user) return <Navigate to="/dashboard" replace />;
```

Net effect: log in once and you stay logged in across closes until you explicitly log out. (The 365-day JWT expiry was never the cause.)

21.2 Why did opening a *second* chat throw a duplicate-key (E11000) error?

Q. The first conversation works; starting a second one (a different buyer, or a friend DM) fails with `E11000 duplicate key`. Why?

A `unique` index over an **array** field is *multikey*, which enforces uniqueness per array element — not per conversation.

Before: the model declared a compound **unique** index on `participants` (an array). Mongo expands that into one index entry per participant, so a user could appear in only one conversation per `listingId` value — the second chat collided and threw `E11000`.

```
conversationSchema.index(
  { listingId: 1, participants: 1 },
  { unique: true }          // multikey on the participants array → wrong
);
```

After: drop the uniqueness; keep a plain lookup index (`getUserConversations` filters by participant). Duplicate conversations are prevented in the application layer instead — `createConversation` does a `findOne`-then-create per participant pair + context.

```
// Lookup only — NOT unique. App layer guarantees no duplicates.
conversationSchema.index({ participants: 1 });
```

Note: the index already existed in the live DB, so it also had to be dropped once on the Atlas cluster — a schema change alone doesn't remove an index that's already built.

21.3 Why did image uploads fail in production?

Q. Listing photos, lost-and-found images and QR uploads worked locally but failed on the deployed backend. Why?

The upload middleware streamed to **Cloudinary**, but production has no Cloudinary credentials configured.

Before: every image route used `cloudinaryStorage`. With no API key/secret in the production environment, the Cloudinary call failed and the whole upload errored out.

```
const storage = createCloudinaryStorage(async (req, file) => ({
  folder: "manit-hub/listings", resource_type: "image", ...
}));   // needs CLOUDINARY_* env vars that prod doesn't have
```

After: a new custom multer engine, `middleware/dataUrlStorage.js`, buffers the file in memory and exposes it as a base64 **data URL** on `file.path` — a drop-in replacement (handlers read `file.path` either way). The image is then stored inline in MongoDB. Tight size caps (2 MB/image, down from 10 MB) plus a client-side resize keep documents well under MongoDB's 16 MB per-document limit.

```
cb(null, {
  path: `data:${file.mimetype};base64,${buffer.toString("base64")}`,
  size: buffer.length,
});
```

21.4 Why didn't a new profile picture show up after saving?

Q. Changing the avatar appeared to succeed but the old picture stuck around. Why?

Same missing-Cloudinary root cause as 21.3, plus the UI never refreshed the cached user.

Before: `uploadAvatar` went through Cloudinary (failing in prod), and even on success `ProfileSettings` didn't update the in-memory/stored user, so the header kept rendering the stale avatar.

After: the avatar route now uses the same inline `dataUrlStorage` engine, and after a successful save `ProfileSettings` calls `login({ user: updatedUser, token })` to refresh the auth context (and durable storage), so the new picture appears immediately everywhere.

End of the Complete Developer Study Guide. Read Parts 1–19 for the narrative, skim Part 20 by folder before an interview, and rehearse Part 19 as flash cards.

Built by students, for students. 🎓